

Transformer para Clasificación Multiclase de Consultas Bancarias

Informe Técnico (Comprensión)

Transfer Learning

Técnica de inteligencia artificial en la que se reutiliza un modelo previamente entrenado en una tarea para resolver otra tarea nueva pero relacionada.

- **Punto de partida:** Se toma un modelo que ya sabe procesar información general.
- **Ajuste fino:** Se modifican las capas finales del modelo para adaptarlas al nuevo problema específico.
- **Entrenamiento rápido:** El modelo ajusta sus parámetros usando muchos menos datos nuevos.

DistilRoBERTa

Modelo de lenguaje basado en la arquitectura Transformer, creado mediante la compresión del modelo RoBERTa-base a través de un proceso llamado destilación de conocimiento. Este se especializa en analizar texto (NLU - Comprensión del Lenguaje Natural).

- **Tamaño reducido:** Tiene 6 capas en lugar de las 12 del modelo original RoBERTa, lo que reduce sus parámetros en un 35% (alrededor de 82 millones de parámetros frente a los 125 millones del original).
- **Utilidad:** Análisis de sentimiento, Búsqueda semántica y Clasificación de textos.

Falcon 7B Instruct

Es un modelo de lenguaje grande (LLM) de código abierto con 7 mil millones de parámetros. Es un modelo causal enfocado en descodificadores (decoder-only) especializado en generar texto (NLG - Generación de Lenguaje Natural).

- **Greedy:** en cada paso elige el token con mayor probabilidad (do_sample=False), en vez de muestrear entre opciones.
- **Greedy_xx:** el valor de max_new_tokens, es decir, el límite de tokens nuevos que puede generar; no cuenta el prompt.
- **Cuantización:** Es una técnica de compresión que reduce el tamaño de los modelos y acelera su funcionamiento bajando la precisión numérica de sus pesos. Los modelos guardan sus parámetros como números largos de 32 o 16 bits (FP32 o FP16).

Implementar dicho metodo ayuda a convertir a formatos más cortos de 8 o 4 bits (INT8 o INT4) usando menos recursos.

Conceptos de AI

- **Temperatura:** Los modelos de IA eligen las palabras (tokens) una por una calculando probabilidades matemáticas. Este valor llamado temperatura actúa como un multiplicador que modifica esas probabilidades antes de tomar una decisión.
 - **Temperatura baja (0.0 a 0.3):** El modelo elige siempre la opción más segura y lógica.
 - **Uso ideal:** Programación, extracción de datos, resúmenes y respuestas factuales.
 - **Temperatura moderada (0.4 a 0.7):** Ofrece un balance natural entre coherencia y variedad.
 - **Uso ideal:** Conversaciones generales y redacción estándar.
 - **Temperatura alta (0.8 a 1.5+):** Aplana las probabilidades y permite que aparezcan palabras menos comunes.
 - **Uso ideal:** Narrativa, lluvia de ideas y escritura creativa.

1. Preparativos

Usa Python 3.10, 3.11 o 3.12. Ejecuta la celda de instalación una vez desde un kernel de esa versión.

En Linux, la celda crea dos entornos en `~/venvs/` y kernels independientes para evitar el conflicto entre las versiones CUDA de TensorFlow y PyTorch.

1. Selecciona **Python (Transformers - TensorFlow GPU)** y vuelve a ejecutar las celdas desde el inicio hasta la clasificación.
2. Selecciona **Python (Transformers - PyTorch GPU)** y ejecuta solo la sección de Falcon; esta carga las predicciones guardadas.

Cambiar de kernel reinicia el estado en memoria. Ejecuta un modelo GPU a la vez para liberar la memoria de la GPU.

```
In [2]: # Ejecutar una vez al abrir el notebook, antes de importar TensorFlow.
# En Linux se crean kernels separados para TensorFlow y PyTorch con GPU.
# En macOS y Windows se instala el conjunto compatible en el entorno actual.
import os
import platform
import subprocess
import sys
from pathlib import Path

# El proyecto se desarrolla con versiones de Python compatibles con TensorFl
```

```

if not (sys.version_info >= (3, 10) and sys.version_info < (3, 13)):
    raise RuntimeError("Este proyecto requiere Python 3.10, 3.11 o 3.12.")

system = platform.system()
machine = platform.machine().lower()
print(f"Sistema operativo detectado: {system} ({machine})")

# Librerías comunes necesarias para EDA, visualización, métricas y carga del
common_requirements = [
    "ipykernel",
    "numpy<2",
    "pandas",
    "matplotlib",
    "seaborn",
    "scipy==1.13.1",
    "scikit-learn==1.7.2",
    "wordcloud",
    "transformers==4.44.2",
    "psutil",
    "truststore>=0.10,<1",
    "tensorflow-datasets==4.9.4",
    "tensorflow-metadata==1.15.0"
]

if system == "Darwin":
    requirements = [
        *common_requirements,
        "tf-keras==2.16.0",
        "torch==2.5.1",
    ]
    # En macOS, la aceleración GPU depende de Apple Silicon (M1/M2/M3).
    # En equipos Intel, TensorFlow puede ejecutarse en CPU pero sin Metal.
    if machine not in {"arm64", "aarch64"}:
        raise RuntimeError(
            "La aceleración GPU de TensorFlow en macOS requiere Apple Silicon."
            "En Intel, usa CPU o un entorno con GPU compatible."
        )
    requirements.extend([
        "tensorflow-macos==2.16.2",
        "tensorflow-metal==1.1.0",
    ])

elif system == "Linux":
    if machine not in {"x86_64", "amd64"}:
        raise RuntimeError("Los entornos GPU de TensorFlow 2.16 y PyTorch 2.

environments = [
    (
        "transformers-tf-gpu",
        "Python (Transformers - TensorFlow GPU)",
        [*common_requirements, "tf-keras==2.16.0", "tensorflow[and-cuda]
    ),
    (
        "transformers-torch-gpu",
        "Python (Transformers - PyTorch GPU)",
        [*common_requirements, "torch==2.5.1", "accelerate>=0.34,<1", "b

```

```

    ),
]
environment_root = Path.home() / ".venvs"
environment_root.mkdir(parents=True, exist_ok=True)

for kernel_name, display_name, packages in environments:
    environment_path = environment_root / kernel_name
    environment_python = environment_path / "bin" / "python"
    if not environment_python.exists():
        subprocess.check_call([sys.executable, "-m", "venv", str(environment_path)])
        subprocess.check_call([str(environment_python), "-m", "pip", "install", "--user", *packages])
    if kernel_name == "transformers-tf-gpu":
        site_packages = list(environment_path.glob("lib/python*/site-packages"))
        if len(site_packages) != 1:
            raise RuntimeError(f"No se pudo localizar site-packages de TensorFlow")
        tensorflow_dir = site_packages[0] / "tensorflow"
        nvidia_dir = site_packages[0] / "nvidia"
        cuda_libraries = sorted(nvidia_dir.glob("*/lib/*.so*"))
        if not cuda_libraries:
            raise RuntimeError(f"No se encontraron bibliotecas CUDA de TensorFlow")
        for library in cuda_libraries:
            link = tensorflow_dir / library.name
            if link.exists() or link.is_symlink():
                if link.resolve() != library.resolve():
                    raise RuntimeError(f"Existe un enlace CUDA incompatible")
            else:
                link.symlink_to(library)

        ptexas_candidates = list(nvidia_dir.glob("cuda_nvcc/bin/ptexas"))
        if len(ptexas_candidates) != 1:
            raise RuntimeError(f"No se pudo localizar ptexas en {nvidia_dir}")
        ptexas_link = environment_path / "bin" / "ptexas"
        if ptexas_link.exists() or ptexas_link.is_symlink():
            if ptexas_link.resolve() != ptexas_candidates[0].resolve():
                raise RuntimeError(f"Existe un enlace ptexas incompatible")
        else:
            ptexas_link.symlink_to(ptexas_candidates[0])
        gpu_check = (
            "import tensorflow as tf; "
            "devices = tf.config.list_physical_devices('GPU'); "
            "print('GPU disponibles:', devices); "
            "assert devices, 'TensorFlow no detectó GPUs; revisa el contenido de la carpeta bin'"
        )
        subprocess.check_call([str(environment_python), "-c", gpu_check])
        subprocess.check_call([
            str(environment_python), "-m", "ipykernel", "install", "--user",
            "--name", kernel_name, "--display-name", display_name,
        ])
        print(f"Kernel registrado: {display_name}")

print("Selecciona el kernel de TensorFlow para DistilRoBERTa y el de PyTorch para DistilGPT2")

elif system == "Windows":
    requirements = [*common_requirements, "tf-keras==2.16.0", "torch==2.5.1"]
    print("Windows nativo: TensorFlow usa CPU; para GPU NVIDIA se requiere W

```

```
else:
    raise RuntimeError(f"Sistema operativo no compatible: {system}")

if system != "Linux":
    os.environ["TF_USE_LEGACY_KERAS"] = "1"
    subprocess.check_call([sys.executable, "-m", "pip", "install", *requirements])
    print(f"Dependencias listas para {system}. Reinicia el kernel antes de i
```

Sistema operativo detectado: Linux (x86_64)
Requirement already satisfied: ipykernel in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (7.3.0)
Requirement already satisfied: numpy<2 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (1.26.4)
Requirement already satisfied: pandas in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (2.3.3)
Requirement already satisfied: matplotlib in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (3.10.9)
Requirement already satisfied: seaborn in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (0.13.2)
Requirement already satisfied: scipy==1.13.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (1.13.1)
Requirement already satisfied: scikit-learn==1.7.2 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (1.7.2)
Requirement already satisfied: wordcloud in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (1.9.6)
Requirement already satisfied: transformers==4.44.2 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (4.44.2)
Requirement already satisfied: psutil in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (7.2.2)
Requirement already satisfied: truststore<1,>=0.10 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (0.10.4)
Requirement already satisfied: tensorflow-datasets==4.9.4 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (4.9.4)
Requirement already satisfied: tensorflow-metadata==1.15.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (1.15.0)
Requirement already satisfied: tf-keras==2.16.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (2.16.0)
Requirement already satisfied: tensorflow[and-cuda]==2.16.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (2.16.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from scikit-learn==1.7.2) (3.7.0)
Requirement already satisfied: joblib>=1.2.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from scikit-learn==1.7.2) (1.6.0)
Requirement already satisfied: requests in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (2.34.2)
Requirement already satisfied: filelock in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (4.0.12)
Requirement already satisfied: pyyaml>=5.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (6.0.3)
Requirement already satisfied: safetensors>=0.4.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (0.8.0)
Requirement already satisfied: tqdm>=4.27 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (4.70.1)
Requirement already satisfied: huggingface-hub<1.0,>=0.23.2 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (0.36.2)
Requirement already satisfied: packaging>=20.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (26.3)
Requirement already satisfied: regex!=2019.12.17 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (2026.9.29)
Requirement already satisfied: tokenizers<0.20,>=0.19 in /home/panchis/.venv

s/transformers-tf-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (0.19.1)
Requirement already satisfied: etils[enp,epath,etree]>=0.9.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (1.13.0)
Requirement already satisfied: toml in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (0.10.2)
Requirement already satisfied: wrapt in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (2.5.0)
Requirement already satisfied: termcolor in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (3.3.0)
Requirement already satisfied: array-record>=0.5.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (0.8.1)
Requirement already satisfied: absl-py in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (2.5.0)
Requirement already satisfied: dm-tree in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (0.1.10)
Requirement already satisfied: protobuf>=3.20 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (3.20.3)
Requirement already satisfied: promise in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (2.3)
Requirement already satisfied: click in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (8.5.0)
Requirement already satisfied: keras>=3.0.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (3.12.4)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (0.7.0)
Requirement already satisfied: google-pasta>=0.1.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (0.2.0)
Requirement already satisfied: h5py>=3.10.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (3.16.0)
Requirement already satisfied: opt-einsum>=2.3.2 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (3.4.0)
Requirement already satisfied: ml-dtypes~=0.3.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (0.3.2)
Requirement already satisfied: six>=1.12.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (1.17.0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (1.84.0)
Requirement already satisfied: libclang>=13.0.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (18.1.1)
Requirement already satisfied: typing-extensions>=3.6.6 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (4.16.0)

Requirement already satisfied: tensorboard<2.17,>=2.16 in /home/panchis/.venv/s/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (2.16.2)

Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (0.37.1)

Requirement already satisfied: setuptools in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (79.0.1)

Requirement already satisfied: astunparse>=1.6.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (1.6.3)

Requirement already satisfied: flatbuffers>=23.5.26 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (25.12.19)

Requirement already satisfied: nvidia-cuda-nvcc-cu12==12.3.107 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (12.3.107)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.3.101 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (12.3.101)

Requirement already satisfied: nvidia-cusolver-cu12==11.5.4.101 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (11.5.4.101)

Requirement already satisfied: nvidia-cufft-cu12==11.0.12.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (11.0.12.1)

Requirement already satisfied: nvidia-cudnn-cu12==8.9.7.29 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (8.9.7.29)

Requirement already satisfied: nvidia-nccl-cu12==2.19.3 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (2.19.3)

Requirement already satisfied: nvidia-cusparse-cu12==12.2.0.103 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (12.2.0.103)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.3.107 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (12.3.107)

Requirement already satisfied: nvidia-cublas-cu12==12.3.4.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (12.3.4.1)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.3.101 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (12.3.101)

Requirement already satisfied: nvidia-curand-cu12==10.3.4.107 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (10.3.4.107)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.3.101 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorflow[and-cuda]==2.16.1) (12.3.101)

Requirement already satisfied: debugpy>=1.6.5 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipykernel) (1.8.22)

Requirement already satisfied: matplotlib-inline>=0.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipykernel) (0.2.2)

Requirement already satisfied: jupyter-client>=8.9.0 in /home/panchis/.venvs/

transformers-tf-gpu/lib/python3.10/site-packages (from ipykernel) (8.10.0)
Requirement already satisfied: ipython>=7.23.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipykernel) (8.39.0)
Requirement already satisfied: nest-asyncio>=1.7.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipykernel) (1.7.3)
Requirement already satisfied: tornado>=6.4.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipykernel) (6.5.10)
Requirement already satisfied: pyzmq>=25 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipykernel) (27.2.0)
Requirement already satisfied: traitlets>=5.4.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipykernel) (5.16.1)
Requirement already satisfied: comm>=0.1.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipykernel) (0.2.3)
Requirement already satisfied: jupyter-core!=6.0.*,>=5.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipykernel) (5.9.1)
Requirement already satisfied: tzdata>=2022.7 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from pandas) (2026.5)
Requirement already satisfied: pytz>=2020.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from pandas) (2026.5)
Requirement already satisfied: python-dateutil>=2.8.2 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: contourpy>=1.0.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from matplotlib) (1.3.2)
Requirement already satisfied: pyparsing>=3 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from matplotlib) (3.3.3)
Requirement already satisfied: kiwisolver>=1.3.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from matplotlib) (1.5.1)
Requirement already satisfied: cyclor>=0.10 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from matplotlib) (4.65.0)
Requirement already satisfied: pillow>=8 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from matplotlib) (12.3.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from astunparse>=1.6.0->tensorflow[and-cuda]==2.16.1) (0.48.0)
Requirement already satisfied: importlib_resources in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from etils[enp,epath,etree]>=0.9.0->tensorflow-datasets==4.9.4) (7.1.0)
Requirement already satisfied: zipp in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from etils[enp,epath,etree]>=0.9.0->tensorflow-datasets==4.9.4) (4.1.1)
Requirement already satisfied: fsspec in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from etils[enp,epath,etree]>=0.9.0->tensorflow-datasets==4.9.4) (2026.9.0)
Requirement already satisfied: einops in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from etils[enp,epath,etree]>=0.9.0->tensorflow-datasets==4.9.4) (0.8.2)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from huggingface-hub<1.0,>=0.23.2->transformers==4.44.2) (1.6.0)
Requirement already satisfied: jedi>=0.16 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipython>=7.23.1->ipykernel) (0.20.0)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in /home/panchi

s/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipython>=7.23.1->ipykernel) (3.0.53)

Requirement already satisfied: pygments>=2.4.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipython>=7.23.1->ipykernel) (2.21.0)

Requirement already satisfied: pexpect>4.3 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipython>=7.23.1->ipykernel) (4.9.0)

Requirement already satisfied: stack_data in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipython>=7.23.1->ipykernel) (0.6.3)

Requirement already satisfied: exceptiongroup in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipython>=7.23.1->ipykernel) (1.3.1)

Requirement already satisfied: decorator in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from ipython>=7.23.1->ipykernel) (5.3.1)

Requirement already satisfied: cloudpickle>=3.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from joblib>=1.2.0->scikit-learn==1.7.2) (3.1.2)

Requirement already satisfied: platformdirs>=2.5 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from jupyter-core!=6.0.*,>=5.1->ipykernel) (4.12.3)

Requirement already satisfied: rich in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from keras>=3.0.0->tensorflow[and-cuda]==2.16.1) (15.0.0)

Requirement already satisfied: optree in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from keras>=3.0.0->tensorflow[and-cuda]==2.16.1) (0.20.0)

Requirement already satisfied: namex in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from keras>=3.0.0->tensorflow[and-cuda]==2.16.1) (0.1.0)

Requirement already satisfied: idna<4,>=2.5 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from requests->transformers==4.44.2) (3.20)

Requirement already satisfied: urllib3<3,>=1.26 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from requests->transformers==4.44.2) (2.8.0)

Requirement already satisfied: certifi>=2023.5.7 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from requests->transformers==4.44.2) (2026.7.22)

Requirement already satisfied: charset_normalizer<4,>=2 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from requests->transformers==4.44.2) (3.5.2)

Requirement already satisfied: werkzeug>=1.0.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorboard<2.17,>=2.16->tensorflow[and-cuda]==2.16.1) (3.1.9)

Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorboard<2.17,>=2.16->tensorflow[and-cuda]==2.16.1) (0.7.2)

Requirement already satisfied: markdown>=2.6.8 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from tensorboard<2.17,>=2.16->tensorflow[and-cuda]==2.16.1) (3.10.3)

Requirement already satisfied: attrs>=18.2.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from dm-tree->tensorflow-datasets==4.9.4) (26.1.0)

Requirement already satisfied: parso<0.9.0,>=0.8.6 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from jedi>=0.16->ipython>=7.23.1->ipykernel) (0.8.7)

Requirement already satisfied: ptyprocess>=0.5 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from pexpect>4.3->ipython>=7.23.1->ipykernel) (0.7.0)

Requirement already satisfied: wcwidth>=0.1.4 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from prompt_toolkit<3.1.0,>=3.0.41->ipython>=7.23.1->ipykernel) (0.9.2)

Requirement already satisfied: markupsafe>=2.1.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from werkzeug>=1.0.1->tensorboard<2.17,>=2.16->tensorflow[and-cuda]==2.16.1) (3.0.4)

Requirement already satisfied: markdown-it-py>=2.2.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from rich->keras>=3.0.0->tensorflow[and-cuda]==2.16.1) (4.2.0)

Requirement already satisfied: pure-eval in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from stack_data->ipython>=7.23.1->ipykernel) (0.2.4)

Requirement already satisfied: executing>=1.2.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from stack_data->ipython>=7.23.1->ipykernel) (2.2.1)

Requirement already satisfied: asttokens>=2.1.0 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from stack_data->ipython>=7.23.1->ipykernel) (3.0.2)

Requirement already satisfied: mdurl~=0.1 in /home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages (from markdown-it-py>=2.2.0->rich->keras>=3.0.0->tensorflow[and-cuda]==2.16.1) (0.1.2)

[notice] A new release of pip is available: 23.0.1 -> 26.2.1

[notice] To update, run: python -m pip install --upgrade pip

2026-10-05 20:17:21.031317: I tensorflow/core/util/port.cc:113] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2026-10-05 20:17:21.053200: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

2026-10-05 20:17:21.419717: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not find TensorRT

2026-10-05 20:17:22.200707: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>

2026-10-05 20:17:22.217263: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>

2026-10-05 20:17:22.218945: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>

```
GPU disponibles: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
Installed kernelspec transformers-tf-gpu in /home/panchis/.local/share/jupyter/kernels/transformers-tf-gpu
Kernel registrado: Python (Transformers - TensorFlow GPU)
Requirement already satisfied: ipykernel in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (7.3.0)
Requirement already satisfied: numpy<2 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (1.26.4)
Requirement already satisfied: pandas in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (2.3.3)
Requirement already satisfied: matplotlib in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (3.10.9)
Requirement already satisfied: seaborn in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (0.13.2)
Requirement already satisfied: scipy==1.13.1 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (1.13.1)
Requirement already satisfied: scikit-learn==1.7.2 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (1.7.2)
Requirement already satisfied: wordcloud in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (1.9.6)
Requirement already satisfied: transformers==4.44.2 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (4.44.2)
Requirement already satisfied: psutil in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (7.2.2)
Requirement already satisfied: truststore<1,>=0.10 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (0.10.4)
Requirement already satisfied: tensorflow-datasets==4.9.4 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (4.9.4)
Requirement already satisfied: tensorflow-metadata==1.15.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (1.15.0)
Requirement already satisfied: torch==2.5.1 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (2.5.1)
Requirement already satisfied: accelerate<1,>=0.34 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (0.34.2)
Requirement already satisfied: bitsandbytes<1,>=0.45 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (0.50.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from scikit-learn==1.7.2) (3.7.0)
Requirement already satisfied: joblib>=1.2.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from scikit-learn==1.7.2) (1.6.0)
Requirement already satisfied: safetensors>=0.4.1 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (0.8.0)
Requirement already satisfied: tqdm>=4.27 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (4.70.1)
Requirement already satisfied: regex!=2019.12.17 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (2026.9.29)
Requirement already satisfied: huggingface-hub<1.0,>=0.23.2 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (0.36.2)
Requirement already satisfied: pyyaml>=5.1 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (6.0.3)
```

Requirement already satisfied: packaging>=20.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (26.3)

Requirement already satisfied: requests in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (2.34.2)

Requirement already satisfied: tokenizers<0.20,>=0.19 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (0.19.1)

Requirement already satisfied: filelock in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from transformers==4.44.2) (4.0.12)

Requirement already satisfied: etils[enp,epath,etree]>=0.9.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (1.13.0)

Requirement already satisfied: protobuf>=3.20 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (3.20.3)

Requirement already satisfied: termcolor in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (3.3.0)

Requirement already satisfied: dm-tree in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (0.1.10)

Requirement already satisfied: wrapt in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (2.5.0)

Requirement already satisfied: absl-py in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (2.5.0)

Requirement already satisfied: array-record>=0.5.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (0.8.1)

Requirement already satisfied: toml in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (0.10.2)

Requirement already satisfied: promise in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (2.3)

Requirement already satisfied: click in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from tensorflow-datasets==4.9.4) (8.5.0)

Requirement already satisfied: nvidia-curand-cu12==10.3.5.147 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (10.3.5.147)

Requirement already satisfied: nvidia-nvtx-cu12==12.4.127 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (12.4.127)

Requirement already satisfied: triton==3.1.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (3.1.0)

Requirement already satisfied: nvidia-cusparse-cu12==12.3.1.170 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (12.3.1.170)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.4.127 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (12.4.127)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.4.127 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (12.4.127)

Requirement already satisfied: sympy==1.13.1 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (1.13.1)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.4.127 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (12.4.127)

his/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (12.4.127)

Requirement already satisfied: nvidia-cudnn-cu12==9.1.0.70 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (9.1.0.70)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.4.127 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (12.4.127)

Requirement already satisfied: fsspec in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (2026.9.0)

Requirement already satisfied: nvidia-cufft-cu12==11.2.1.3 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (11.2.1.3)

Requirement already satisfied: jinja2 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (3.1.6)

Requirement already satisfied: nvidia-nccl-cu12==2.21.5 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (2.21.5)

Requirement already satisfied: nvidia-cublas-cu12==12.4.5.8 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (12.4.5.8)

Requirement already satisfied: nvidia-cusolver-cu12==11.6.1.9 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (11.6.1.9)

Requirement already satisfied: networkx in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (3.4.2)

Requirement already satisfied: typing-extensions>=4.8.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from torch==2.5.1) (4.16.0)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from sympy==1.13.1->torch==2.5.1) (1.3.0)

Requirement already satisfied: matplotlib-inline>=0.1 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipykernel) (0.2.2)

Requirement already satisfied: ipython>=7.23.1 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipykernel) (8.39.0)

Requirement already satisfied: traitlets>=5.4.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipykernel) (5.16.1)

Requirement already satisfied: tornado>=6.4.1 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipykernel) (6.5.10)

Requirement already satisfied: comm>=0.1.1 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipykernel) (0.2.3)

Requirement already satisfied: debugpy>=1.6.5 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipykernel) (1.8.22)

Requirement already satisfied: nest-asyncio2>=1.7.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipykernel) (1.7.3)

Requirement already satisfied: jupyter-core!=6.0.*,>=5.1 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipykernel) (5.9.1)

Requirement already satisfied: pyzmq>=25 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipykernel) (27.2.0)

Requirement already satisfied: jupyter-client>=8.9.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipykernel) (8.10.0)

Requirement already satisfied: python-dateutil>=2.8.2 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from pandas) (2.9.0.post0)

st0)

Requirement already satisfied: pytz>=2020.1 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from pandas) (2026.5)

Requirement already satisfied: tzdata>=2022.7 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from pandas) (2026.5)

Requirement already satisfied: pyparsing>=3 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from matplotlib) (3.3.3)

Requirement already satisfied: fonttools>=4.22.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from matplotlib) (4.65.0)

Requirement already satisfied: contourpy>=1.0.1 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from matplotlib) (1.3.2)

Requirement already satisfied: pillow>=8 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from matplotlib) (12.3.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from matplotlib) (1.5.1)

Requirement already satisfied: cyclor>=0.10 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from matplotlib) (0.12.1)

Requirement already satisfied: einops in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from etils[enp,epath,etree]>=0.9.0->tensorflow-datasets==4.9.4) (0.8.2)

Requirement already satisfied: zipp in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from etils[enp,epath,etree]>=0.9.0->tensorflow-datasets==4.9.4) (4.1.1)

Requirement already satisfied: importlib_resources in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from etils[enp,epath,etree]>=0.9.0->tensorflow-datasets==4.9.4) (7.1.0)

Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from huggingface-hub<1.0,>=0.23.2->transformers==4.44.2) (1.6.0)

Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipython>=7.23.1->ipykernel) (3.0.53)

Requirement already satisfied: pexpect>4.3 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipython>=7.23.1->ipykernel) (4.9.0)

Requirement already satisfied: stack_data in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipython>=7.23.1->ipykernel) (0.6.3)

Requirement already satisfied: exceptiongroup in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipython>=7.23.1->ipykernel) (1.3.1)

Requirement already satisfied: decorator in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipython>=7.23.1->ipykernel) (5.3.1)

Requirement already satisfied: jedi>=0.16 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipython>=7.23.1->ipykernel) (0.20.0)

Requirement already satisfied: pygments>=2.4.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from ipython>=7.23.1->ipykernel) (2.21.0)

Requirement already satisfied: cloudpickle>=3.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from joblib>=1.2.0->scikit-learn==1.7.2) (3.1.2)

Requirement already satisfied: platformdirs>=2.5 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from jupyter-core!=6.0.*,>=5.1->ipykernel) (4.12.3)

Requirement already satisfied: six>=1.5 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

Requirement already satisfied: certifi>=2023.5.7 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from requests->transformers==4.44.2) (2026.7.22)

Requirement already satisfied: charset_normalizer<4,>=2 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from requests->transformers==4.44.2) (3.5.2)

Requirement already satisfied: idna<4,>=2.5 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from requests->transformers==4.44.2) (3.20)

Requirement already satisfied: urllib3<3,>=1.26 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from requests->transformers==4.44.2) (2.8.0)

Requirement already satisfied: attrs>=18.2.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from dm-tree->tensorflow-datasets==4.9.4) (26.1.0)

Requirement already satisfied: MarkupSafe>=2.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from jinja2->torch==2.5.1) (3.0.4)

Requirement already satisfied: parso<0.9.0,>=0.8.6 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from jedi>=0.16->ipython>=7.23.1->ipykernel) (0.8.7)

Requirement already satisfied: ptyprocess>=0.5 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from pexpect>4.3->ipython>=7.23.1->ipykernel) (0.7.0)

Requirement already satisfied: wcwidth>=0.1.4 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from prompt_toolkit<3.1.0,>=3.0.41->ipython>=7.23.1->ipykernel) (0.9.2)

Requirement already satisfied: pure-eval in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from stack_data->ipython>=7.23.1->ipykernel) (0.2.4)

Requirement already satisfied: asttokens>=2.1.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from stack_data->ipython>=7.23.1->ipykernel) (3.0.2)

Requirement already satisfied: executing>=1.2.0 in /home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages (from stack_data->ipython>=7.23.1->ipykernel) (2.2.1)

[notice] A new release of pip is available: 23.0.1 -> 26.2.1

[notice] To update, run: `pip install --upgrade pip`

Installed kernelspec transformers-torch-gpu in /home/panchis/.local/share/jupyter/kernels/transformers-torch-gpu

Kernel registrado: Python (Transformers - PyTorch GPU)

Selecciona el kernel de TensorFlow para DistilRoBERTa y el de PyTorch para Falcon.

2. Análisis Exploratorio de Datos (EDA)

El dataset **PolyAI/banking77** contiene 13,083 consultas bancarias breves en inglés, etiquetadas en 77 intenciones (por ejemplo: *activate my card*, *age limit*, *apple pay or google pay*, *atm support*, *automatic top up*, *balance not updated after bank transfer*).

Partición	Muestras
Entrenamiento (train)	10,003
Prueba (test)	3,080

Fuente: <https://huggingface.co/datasets/PolyAI/banking77>. Los CSV se descargan del repositorio original de PolyAI, que es la misma fuente que usa el script de carga de Hugging Face.

```
In [5]: import re
from collections import Counter

import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from wordcloud import STOPWORDS as WORDCLOUD_STOPWORDS, WordCloud

sns.set_theme(style="whitegrid")

# Se conservan las particiones oficiales; la concatenación se usa solo para
BASE_URL = "https://raw.githubusercontent.com/PolyAI-LDN/task-specific-datas
train_df = pd.read_csv(f"{BASE_URL}/train.csv").assign(split="train")
test_df = pd.read_csv(f"{BASE_URL}/test.csv").assign(split="test")
df = pd.concat([train_df, test_df], ignore_index=True)

print(f"Train: {len(train_df):,} | Test: {len(test_df):,} | Total: {len(df):,}")
print(f"Número de categorías: {df['category'].nunique()}")
print(f"Valores nulos:\n{df.isna().sum()}")
print(f"Consultas duplicadas: {df.duplicated(subset='text').sum()}")
df.head()
```

```
Train: 10,003 | Test: 3,080 | Total: 13,083
Número de categorías: 77
Valores nulos:
text      0
category  0
split     0
dtype: int64
Consultas duplicadas: 0
```

```
Out [5]:
```

	text	category	split
0	I am still waiting on my card?	card_arrival	train
1	What can I do if my card still hasn't arrived ...	card_arrival	train
2	I have been waiting over a week. Is the card s...	card_arrival	train
3	Can I track my card while it is in the process...	card_arrival	train
4	How do I know if I will get my card, or if it ...	card_arrival	train

2.1 Longitud y número de palabras por consulta

```
In [6]: df["char_length"] = df["text"].str.len()
df["word_count"] = df["text"].str.split().str.len()

df[["text", "char_length", "word_count"]].head(10)
```

```
Out [6]:
```

	text	char_length	word_count
0	I am still waiting on my card?	30	7
1	What can I do if my card still hasn't arrived ...	60	13
2	I have been waiting over a week. Is the card s...	58	12
3	Can I track my card while it is in the process...	59	13
4	How do I know if I will get my card, or if it ...	54	15
5	When did you send me my new card?	33	8
6	Do you have info about the card on delivery?	44	9
7	What do I do if I still have not received my n...	54	13
8	Does the package with my card have tracking?	44	8
9	I ordered my card but it still isn't here	41	9

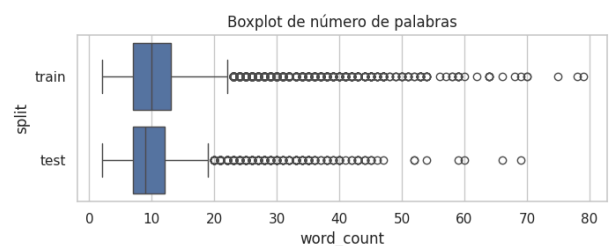
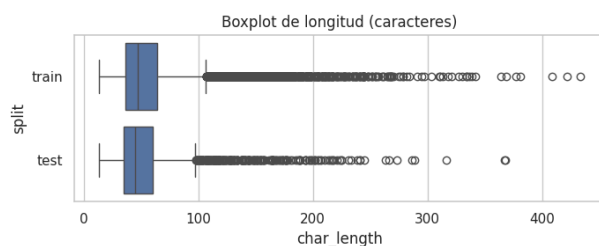
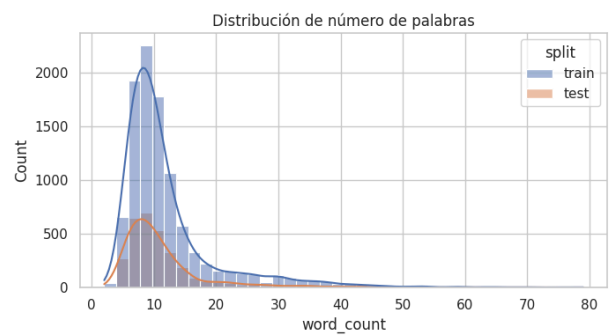
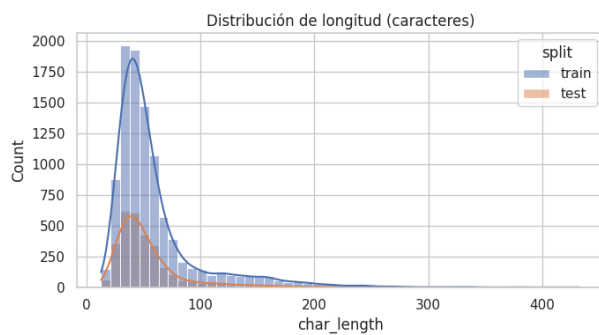
2.2 Estadísticas descriptivas de longitud y número de palabras

```
In [7]: stats = df.groupby("split")[["char_length", "word_count"]].describe().T
stats["total"] = df[["char_length", "word_count"]].describe().T.stack()
display(stats.round(2))

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
sns.histplot(data=df, x="char_length", hue="split", bins=50, kde=True, ax=axes[0])
axes[0].set_title("Distribución de longitud (caracteres)")
sns.histplot(data=df, x="word_count", hue="split", bins=40, kde=True, ax=axes[1])
axes[1].set_title("Distribución de número de palabras")
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(1, 2, figsize=(14, 3))
sns.boxplot(data=df, x="char_length", y="split", ax=axes[0])
axes[0].set_title("Boxplot de longitud (caracteres)")
sns.boxplot(data=df, x="word_count", y="split", ax=axes[1])
axes[1].set_title("Boxplot de número de palabras")
plt.tight_layout()
plt.show()
```

	split	test	train	total
char_length	count	3080.00	10003.00	13083.00
	mean	54.23	59.47	58.24
	std	34.66	40.87	39.56
	min	13.00	13.00	13.00
	25%	35.00	36.00	36.00
	50%	45.00	47.00	46.00
	75%	60.00	64.00	63.00
	max	368.00	433.00	433.00
word_count	count	3080.00	10003.00	13083.00
	mean	10.95	11.95	11.71
	std	6.69	7.89	7.64
	min	2.00	2.00	2.00
	25%	7.00	7.00	7.00
	50%	9.00	10.00	10.00
	75%	12.00	13.00	13.00
	max	69.00	79.00	79.00



2.3 Limpieza básica del texto

1. Conversión a minúsculas.
2. Eliminación de caracteres especiales (números, signos de puntuación, símbolos) y stopwords en inglés.

Se usa la lista de stopwords incluida en `wordcloud`, conservando negaciones (`not` , `no` , `nor` , `cannot`) porque cambian la intención de la consulta (p. ej., *card not working*).

```
In [8]: NEGATIONS = {"not", "no", "nor", "cannot"}
# Restos de contracciones tras quitar apóstrofes (i'm -> im, i've -> ive).
CONTRACTION_LEFTOVERS = {"im", "ive", "id", "ill", "youre", "theyre", "thats"}
# Se conservan las negaciones porque cambian la intención; el texto limpio s
STOPWORDS = (set(WORDCLOUD_STOPWORDS) - NEGATIONS) | CONTRACTION_LEFTOVERS

def clean_text(text: str) -> str:
    text = text.lower()
    text = text.replace("'", "").replace('"', "")
    # Reemplazar, en vez de concatenar, evita unir accidentalmente palabras
    text = re.sub(r"^[a-z\s]", " ", text)
    tokens = [tok for tok in text.split() if tok not in STOPWORDS and len(tok) > 2]
    return " ".join(tokens)

df["clean_text"] = df["text"].apply(clean_text)
df["clean_word_count"] = df["clean_text"].str.split().str.len()

print(f"Consultas vacías tras la limpieza: {(df['clean_word_count'] == 0).sum()}")
print(f"Promedio de palabras antes: {df['word_count'].mean():.2f} | después: {df['clean_word_count'].mean():.2f}")
df[["text", "clean_text"]].sample(10, random_state=42)
```

Consultas vacías tras la limpieza: 1
Promedio de palabras antes: 11.71 | después: 5.24

Out [8]:

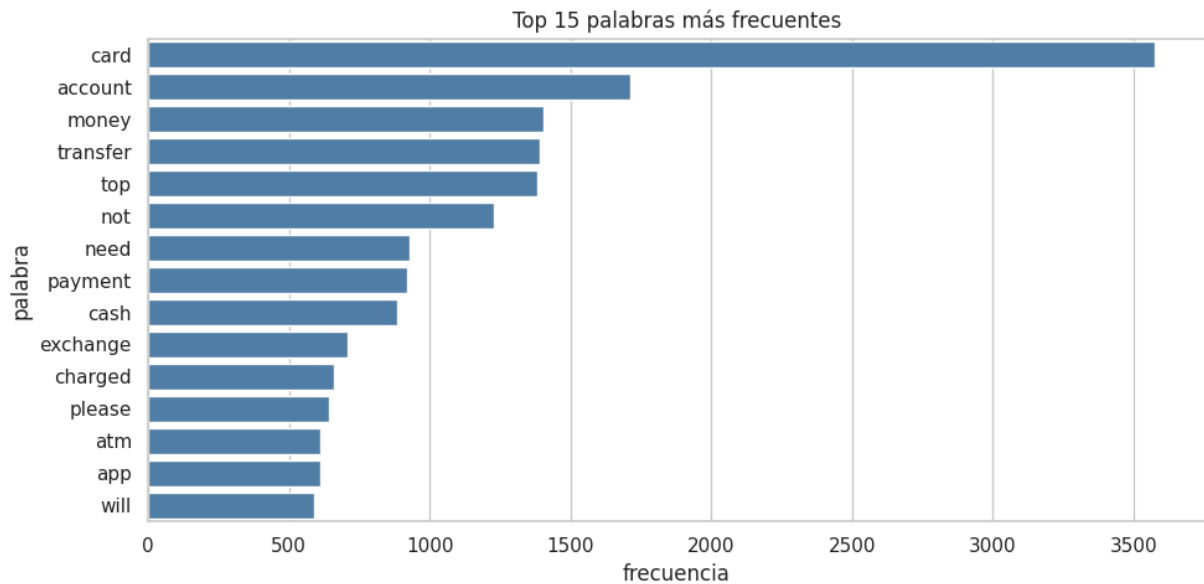
	text	clean_text
353	Please explain the exchange rates.	please explain exchange rates
10210	I got some cash of an ATM earlier but this sho...	got cash atm earlier shows pending app still p...
5361	I just replaced my phone but how do I use my c...	replaced phone use card information
7051	I wasn't able to do a transfer to an account	wasnt able transfer account
3617	I don't recognize a charge on my statement.	dont recognize charge statement
10577	How do I check my security settings to allow c...	check security settings allow contactless pay
6087	What is the average pending time for a transac...	average pending time transaction
2267	is there a limit for top up?	limit top
7712	Why do we need to verify a top-up card?	need verify top card
5576	I have been doing some online shopping, but wh...	online shopping checking keeps getting decline...

2.4 Las 15 palabras más frecuentes

```
In [9]: word_counts = Counter(" ".join(df["clean_text"]).split())
top_words = pd.DataFrame(word_counts.most_common(15), columns=["palabra", "frecuencia"])
display(top_words)

plt.figure(figsize=(10, 5))
sns.barplot(data=top_words, x="frecuencia", y="palabra", color="steelblue")
plt.title("Top 15 palabras más frecuentes")
plt.tight_layout()
plt.show()
```

	palabra	frecuencia
0	card	3578
1	account	1714
2	money	1405
3	transfer	1391
4	top	1381
5	not	1230
6	need	927
7	payment	919
8	cash	884
9	exchange	708
10	charged	660
11	please	643
12	atm	612
13	app	611
14	will	589



2.5 Los 10 bigramas y 10 trigramas más frecuentes

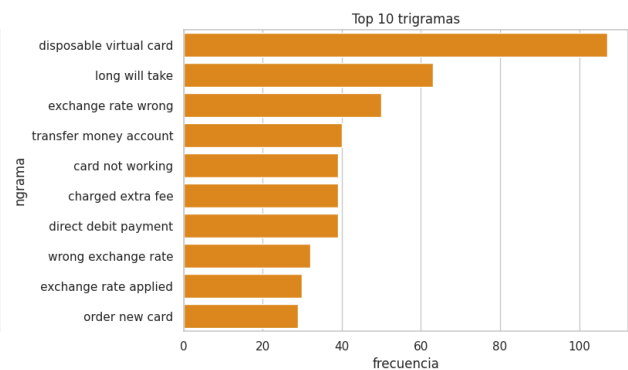
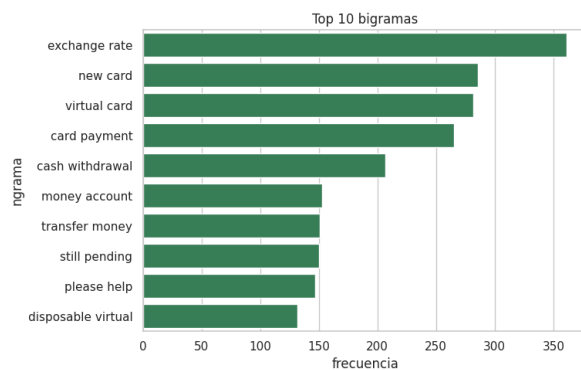
```
In [10]: def top_ngrams(texts: pd.Series, n: int, top_k: int = 10) -> pd.DataFrame:
# Contar ventanas contiguas de n palabras conserva expresiones cuyo sent
counter = Counter()
for text in texts:
    tokens = text.split()
    counter.update(" ".join(tokens[i:i + n]) for i in range(len(tokens)
return pd.DataFrame(counter.most_common(top_k), columns=["ngrama", "frec

top_bigrams = top_ngrams(df["clean_text"], n=2)
top_trigrams = top_ngrams(df["clean_text"], n=3)
display(top_bigrams, top_trigrams)

fig, axes = plt.subplots(1, 2, figsize=(16, 5))
sns.barplot(data=top_bigrams, x="frecuencia", y="ngrama", color="seagreen",
axes[0].set_title("Top 10 bigramas")
sns.barplot(data=top_trigrams, x="frecuencia", y="ngrama", color="darkorange",
axes[1].set_title("Top 10 trigramas")
plt.tight_layout()
plt.show()
```

	ngrama	frecuencia
0	exchange rate	361
1	new card	286
2	virtual card	282
3	card payment	265
4	cash withdrawal	207
5	money account	153
6	transfer money	151
7	still pending	150
8	please help	147
9	disposable virtual	132

	ngrama	frecuencia
0	disposable virtual card	107
1	long will take	63
2	exchange rate wrong	50
3	transfer money account	40
4	card not working	39
5	charged extra fee	39
6	direct debit payment	39
7	wrong exchange rate	32
8	exchange rate applied	30
9	order new card	29



2.6 Nube de palabras


```

    )

fig, ax = plt.subplots(figsize=(12, 18))
class_counts[["train", "test"]].plot.barh(stacked=True, ax=ax, color=["steel", "steel"])
ax.axvline(class_counts["train"].mean(), color="red", linestyle="--", label="media")
ax.invert_yaxis()
ax.set_title("Número de muestras por categoría (train + test)")
ax.set_xlabel("Muestras")
ax.legend()
plt.tight_layout()
plt.show()

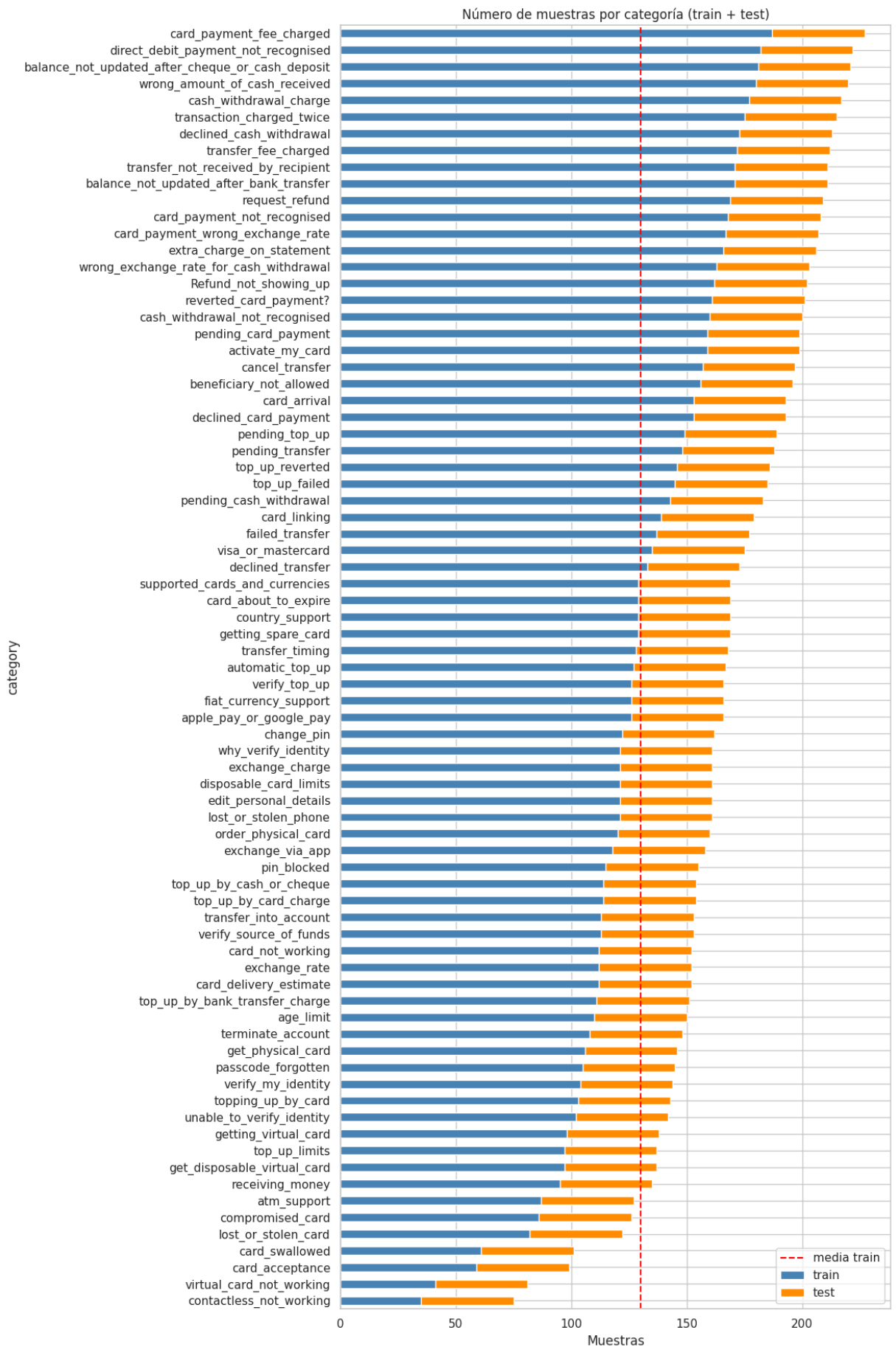
display(class_counts.head(5), class_counts.tail(5))

```

```

[train] min: 35 (contactless_not_working) | max: 187 (card_payment_fee_charge
d) | media: 129.9 | desv. est.: 32.9 | imbalance ratio: 5.34 | CV: 25.36%
[test] min: 40 (card_payment_fee_charged) | max: 40 (card_payment_fee_charge
d) | media: 40.0 | desv. est.: 0.0 | imbalance ratio: 1.00 | CV: 0.00%

```



	split	test	train
category			
card_payment_fee_charged		40	187
direct_debit_payment_not_recognised		40	182
balance_not_updated_after_cheque_or_cash_deposit		40	181
wrong_amount_of_cash_received		40	180
cash_withdrawal_charge		40	177

	split	test	train
category			
lost_or_stolen_card		40	82
card_swallowed		40	61
card_acceptance		40	59
virtual_card_not_working		40	41
contactless_not_working		40	35

2.8 Conclusiones del análisis exploratorio

Calidad de los datos

- Se confirman 13,083 consultas (10,003 train / 3,080 test) y 77 categorías, sin valores nulos ni textos duplicados.

Longitud de las consultas

- Las consultas son breves: en promedio, ~58 caracteres y ~11.7 palabras (mediana de 46 caracteres y 10 palabras).
- La distribución presenta asimetría positiva marcada: el 75 % de las consultas tiene 13 palabras o menos, aunque existen valores atípicos de hasta 79 palabras (433 caracteres).
- Train y test tienen distribuciones muy parecidas (test ligeramente más corto), por lo que el conjunto de prueba es representativo.
- Implicación para el Transformer: una longitud máxima de secuencia de 64 tokens cubre prácticamente todas las consultas sin truncar información relevante, lo que reduce memoria y tiempo de entrenamiento.

Vocabulario y limpieza

- Tras la limpieza, el promedio baja de 11.7 a 5.2 palabras: más de la mitad de cada consulta son stopwords. Solo una consulta queda vacía.

- Las palabras más frecuentes (*card, account, money, transfer, top, payment, cash, exchange, charged, atm*) reflejan el dominio bancario: tarjetas, transferencias, recargas (*top up*) y comisiones.
- La negación *not* aparece entre las 6 palabras más frecuentes, lo que justifica conservarla: distingue intenciones como *card_not_working* o *contactless_not_working*.
- Los bigramas y trigramas (*exchange rate, virtual card, disposable virtual card, card not working, charged extra fee, wrong exchange rate*) están muy alineados con los nombres de las categorías, por lo que el contexto de varias palabras es clave para separarlas.
- Muchas clases comparten vocabulario (p. ej., *card_payment_fee_charged* vs. *transfer_fee_charged, exchange_rate* vs. *card_payment_wrong_exchange_rate*). Esto dificulta la clasificación con bolsas de palabras y favorece modelos contextuales como los Transformers.
- La limpieza agresiva (eliminar stopwords) es útil para el EDA, pero para el fine-tuning conviene usar el texto original con el tokenizador del modelo preentrenado, ya que las palabras funcionales y la puntuación aportan contexto.

Balance de clases

- **Test está perfectamente balanceado:** 40 muestras por clase.
- **Train está moderadamente desbalanceado:** de 35 (*contactless_not_working*) a 187 (*card_payment_fee_charged*) muestras, con un *imbalance ratio* de 5.34 y un coeficiente de variación de ~25 %.
- Las clases minoritarias (*contactless_not_working, virtual_card_not_working, card_acceptance, card_swallowed*) tienen menos de la mitad de la media (~130), por lo que es probable que el modelo tenga menor *recall* en ellas.
- Recomendación: usar pesos de clase (`class_weight`) o sobremuestreo de las clases minoritarias y evaluar con métricas macro (F1-macro) además de *accuracy*, aprovechando que el test balanceado permite comparar el rendimiento por clase de forma justa.

3. Clasificación con Transformer (DistilRoBERTa)

Se ajusta `distilbert/distilroberta-base` para reconocer las 77 intenciones de BANKING77 con TensorFlow. Se usan los textos originales, no `clean_text`: las negaciones, la puntuación y las palabras funcionales aportan contexto.

Conexión explícita entre EDA y modelo. El EDA encontró un *imbalance ratio* de **5.34** en train (35–187 ejemplos por clase), mientras que test tiene exactamente 40 por clase. Por eso `USE_CLASS_WEIGHTS=True` compensa la frecuencia de cada clase usando únicamente el subconjunto de ajuste; el F1-macro y el reporte por clase permiten comprobar si esto mejora las clases minoritarias. El EDA también mostró negaciones frecuentes y expresiones de varias

palabras compartidas entre intenciones. En consecuencia, no se eliminan stopwords ni se alimenta `clean_text` al Transformer: se conservan las consultas originales y se revisan las confusiones dirigidas para distinguir conceptos próximos.

El entrenamiento usa el 85 % de **train** y la validación estratificada el 15 % restante. **Test** se reserva para la evaluación final, sin participar en la selección de épocas, pesos de clase ni longitud de secuencia. Ejecutar primero Preparativos, reiniciar el kernel y cargar los datos del EDA.

3.1 Tokenización y preparación

El tokenizer BPE de DistilRoBERTa genera `input_ids` y `attention_mask`. La longitud máxima se fija con el percentil 99 de los tokens del subconjunto de entrenamiento, incluyendo tokens especiales; se informa cuántos textos se truncan en cada partición. El vocabulario y la asignación de etiquetas se conservan para inferencia.

```
In [ ]: import os
import sys
from pathlib import Path
import truststore

# Usa las autoridades de confianza del sistema sin desactivar la verificació
# La descarga del tokenizer falló por verificación de certificados HTTPS; se
truststore.inject_into_ssl()

os.environ["TF_USE_LEGACY_KERAS"] = "1"
if "keras" in sys.modules and str(sys.modules["keras"].__version__).startswith("2."):
    raise RuntimeError("Reinicia el kernel y ejecuta Preparativos antes de i

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
import tf_keras as keras
from IPython.display import Markdown, display
from transformers import AutoTokenizer, TFAutoModelForSequenceClassification

SEED = 42
MODEL_NAME = "distilbert/distilroberta-base"
MODEL_REVISION = "fb53ab8802853c8e4fbdabcd0529f21fc6f459b2b"
BATCH_SIZE = 8
EPOCHS = 5
LEARNING_RATE = 2e-5
VALIDATION_FRACTION = 0.15
USE_CLASS_WEIGHTS = True
OUTPUT_DIR = Path("distilroberta_banking77")
keras.utils.set_random_seed(SEED)

assert len(train_df) == 10003 and len(test_df) == 3080
assert train_df[["text", "category"]].notna().all().all()
```

```

assert test_df[["text", "category"]].notna().all().all()
class_names = sorted(train_df["category"].unique().tolist())
assert len(class_names) == 77
assert set(test_df["category"]) == set(class_names)
label2id = {name: index for index, name in enumerate(class_names)}
id2label = {index: name for name, index in label2id.items()}

rng = np.random.default_rng(SEED)
fit_indices, validation_indices = [], []
# Muestrear dentro de cada etiqueta conserva su proporción tanto en ajuste c
for category in class_names:
    indices = np.flatnonzero(train_df["category"].to_numpy() == category)
    rng.shuffle(indices)
    validation_size = max(1, int(round(len(indices) * VALIDATION_FRACTION)))
    validation_indices.extend(indices[:validation_size])
    fit_indices.extend(indices[validation_size:])

fit_df = train_df.iloc[fit_indices].reset_index(drop=True)
validation_df = train_df.iloc[validation_indices].reset_index(drop=True)
evaluation_df = test_df.reset_index(drop=True).copy()
assert set(fit_indices).isdisjoint(validation_indices)
assert len(fit_df) + len(validation_df) == len(train_df)
print(f"Entrenamiento: {len(fit_df)} | Validación: {len(validation_df)} | Te
print("GPU disponibles:", tf.config.list_physical_devices("GPU"))

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME, revision=MODEL_REVISIO
# Elegir la longitud solo con train evita usar información de validación o t
fit_token_lengths = np.array([
    len(tokens) for tokens in tokenizer(fit_df["text"].tolist(), truncation=
])
MAX_LENGTH = min(tokenizer.model_max_length, int(np.ceil(np.percentile(fit_t
print(f"Longitud máxima: {MAX_LENGTH} tokens (percentil 99 de entrenamiento)

def encode_frame(frame):
    # El padding uniforme facilita el batching; truncation limita memoria y
    return dict(tokenizer(
        frame["text"].tolist(), padding="max_length", truncation=True,
        max_length=MAX_LENGTH, return_tensors="np",
    ))

fit_encoding = encode_frame(fit_df)
validation_encoding = encode_frame(validation_df)
test_encoding = encode_frame(evaluation_df)
fit_labels = fit_df["category"].map(label2id).to_numpy(dtype=np.int32)
validation_labels = validation_df["category"].map(label2id).to_numpy(dtype=np
test_labels = evaluation_df["category"].map(label2id).to_numpy(dtype=np.int3

for split_name, frame in [("train", fit_df), ("validation", validation_df),
    lengths = np.array([len(tokens) for tokens in tokenizer(frame["text"].to
    print(f"{split_name}: {np.mean(lengths > MAX_LENGTH):.2%} de textos trun
    if split_name == "test":
        test_token_lengths = lengths

example = tokenizer(fit_df.loc[0, "text"], truncation=True, max_length=MAX_L

```

```

print("Texto:", fit_df.loc[0, "text"])
print("Tokens:", tokenizer.convert_ids_to_tokens(example["input_ids"]))

def make_dataset(encoding, labels=None, shuffle=False):
    # El dataset sin etiquetas se usa para inferencia; el de ajuste mezcla e
    tensors = encoding if labels is None else (encoding, labels)
    dataset = tf.data.Dataset.from_tensor_slices(tensors)
    if shuffle:
        dataset = dataset.shuffle(len(labels), seed=SEED, reshuffle_each_ite
    return dataset.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

fit_dataset = make_dataset(fit_encoding, fit_labels, shuffle=True)
validation_dataset = make_dataset(validation_encoding, validation_labels)
test_dataset = make_dataset(test_encoding)

```

Entrenamiento: 8505 | Validación: 1498 | Test: 3080

GPU disponibles: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Longitud máxima: 51 tokens (percentil 99 de entrenamiento)

2026-10-05 12:49:39.266782: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>

2026-10-05 12:49:39.283934: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>

2026-10-05 12:49:39.284940: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>

/home/panchis/.venvs/transformers-tf-gpu/lib/python3.10/site-packages/transformers/tokenization_utils_base.py:1601: FutureWarning: `clean\_up\_tokenization\_spaces` was not set. It will be set to `True` by default. This behavior will be deprecated in transformers v4.45, and will be then set to `False` by default. For more details check this issue: <https://github.com/huggingface/transformers/issues/31884>

warnings.warn(

train: 0.95% de textos truncados

validation: 1.07% de textos truncados

test: 0.55% de textos truncados

Texto: No refund showing in my statement.

Tokens: ['<s>', 'No', 'Ġrefund', 'Ġshowing', 'Ġin', 'Ġmy', 'Ġstatement', '.', '</s>']

```
2026-10-05 12:49:39.727785: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-05 12:49:39.729422: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-05 12:49:39.730130: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-05 12:49:39.822606: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-05 12:49:39.823387: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-05 12:49:39.823967: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-05 12:49:39.824565: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1928] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 13044 MB memory:  -> device: 0, name: NVIDIA GeForce RTX 4080, pci bus id: 0000:01:00.0, compute capability: 8.9
```

3.2 Fine-tuning del modelo preentrenado

Se sustituye la cabeza de preentrenamiento por un clasificador de 77 etiquetas y se ajustan **todas las capas**. Es normal recibir un aviso de pesos nuevos en el clasificador: estos se aprenden durante el fine-tuning.

Configuración inicial: batch de 8, hasta 5 épocas, AdamW con tasa de aprendizaje `2e-5`, warmup del 10 %, decaimiento de pesos y clipping de gradientes. Los pesos de clase se calculan solo con el subconjunto de entrenamiento. La parada temprana restaura la época con menor pérdida de validación (sin ponderar). Estos hiperparámetros son un punto de partida, no una garantía de rendimiento.

Esta celda descarga los pesos de TensorFlow del modelo (~487 MB) y puede tardar considerablemente en CPU. Si falta memoria, reducir `BATCH_SIZE` ; no modificar test para elegir los hiperparámetros.

```

In [15]: import os
from huggingface_hub import constants as hub_constants

# Xet es el sistema que utiliza Hugging Face para almacenar y descargar archivos
# Cuando está instalado hf_xet, Hugging Face puede usarlo automáticamente.

# En caso de que la ruta de Xet falle con 403 Forbidden y la descarga HTTP c
os.environ["HF_HUB_DISABLE_XET"] = "1"
hub_constants.HF_HUB_DISABLE_XET = True

model = TFAutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME, revision=MODEL_REVISION, num_labels=len(class_names),
    id2label=id2label, label2id=label2id,
)
model.trainable = True
steps_per_epoch = int(np.ceil(len(fit_df) / BATCH_SIZE))
training_steps = steps_per_epoch * EPOCHS
optimizer, learning_rate_schedule = create_optimizer(
    init_lr=LEARNING_RATE, num_train_steps=training_steps,
    num_warmup_steps=int(training_steps * 0.1), weight_decay_rate=0.01,
    adam_global_clipnorm=1.0,
)
model.compile(
    optimizer=optimizer,
    # El modelo entrega logits sin normalizar; from_logits=True evita aplica
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=[keras.metrics.SparseCategoricalAccuracy(name="accuracy")],
)
# Inversa de frecuencia normalizada: el peso medio por clase queda aproximad
# Se calcula en el subconjunto de ajuste para no incorporar información de v
fit_support = np.bincount(fit_labels, minlength=len(class_names))
class_weights = {
    index: float(len(fit_labels) / (len(class_names) * support))
    for index, support in enumerate(fit_support)
}
training_history = model.fit(
    fit_dataset, validation_data=validation_dataset, epochs=EPOCHS,
    class_weight=class_weights if USE_CLASS_WEIGHTS else None,
    callbacks=[
        keras.callbacks.EarlyStopping(monitor="val_loss", patience=2, restor
        keras.callbacks.TerminateOnNaN(),
    ],
)
assert np.isfinite(training_history.history["val_loss"]).all(), "Pérdida no
best_epoch = int(np.argmax(training_history.history["val_loss"]) + 1)
print(f"Mejor época por pérdida de validación: {best_epoch}")

OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
model.save_pretrained(str(OUTPUT_DIR))
tokenizer.save_pretrained(str(OUTPUT_DIR))
import json
(OUTPUT_DIR / "training_config.json").write_text(json.dumps({
    "model": MODEL_NAME, "revision": MODEL_REVISION, "seed": SEED,
    "max_length": MAX_LENGTH, "batch_size": BATCH_SIZE,

```

```

    "learning_rate": LEARNING_RATE, "best_epoch": best_epoch,
    "class_weights": USE_CLASS_WEIGHTS,
}, indent=2), encoding="utf-8")

history_df = pd.DataFrame(training_history.history)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for metric_name, axis in zip(["loss", "accuracy"], axes):
    axis.plot(history_df.index + 1, history_df[metric_name], label="train")
    axis.plot(history_df.index + 1, history_df[f"val_{metric_name}"], label="val")
    axis.set(title=metric_name, xlabel="Época")
    axis.legend()
plt.tight_layout()
plt.show()

```

All PyTorch model weights were used when initializing TFRobertaForSequenceClassification.

Some weights or buffers of the TF 2.0 model TFRobertaForSequenceClassification were not initialized from the PyTorch model and are newly initialized: ['classifier.dense.weight', 'classifier.dense.bias', 'classifier.out_proj.weight', 'classifier.out_proj.bias']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

Epoch 1/5

WARNING:tensorflow:AutoGraph could not transform <function infer_framework at 0x7f0da6ffe290> and will run it as-is.

Cause: for/else statement not yet supported

To silence this warning, decorate the function with @tf.autograph.experimental.do_not_convert

WARNING: AutoGraph could not transform <function infer_framework at 0x7f0da6ffe290> and will run it as-is.

Cause: for/else statement not yet supported

To silence this warning, decorate the function with @tf.autograph.experimental.do_not_convert

1064/1064 [=====] - 53s 33ms/step - loss: 3.1625 - accuracy: 0.4178 - val_loss: 1.4141 - val_accuracy: 0.8091

Epoch 2/5

1064/1064 [=====] - 32s 30ms/step - loss: 1.0398 - accuracy: 0.8556 - val_loss: 0.5639 - val_accuracy: 0.8905

Epoch 3/5

1064/1064 [=====] - 33s 31ms/step - loss: 0.4606 - accuracy: 0.9193 - val_loss: 0.3626 - val_accuracy: 0.9126

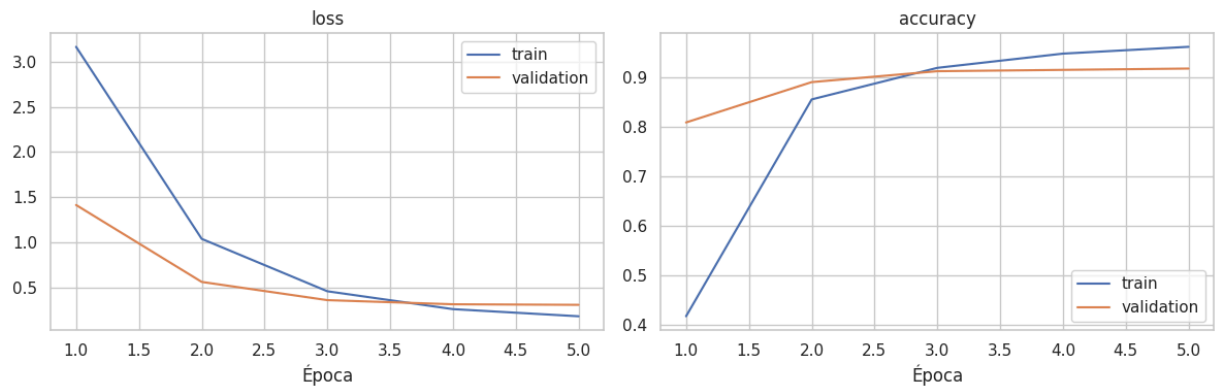
Epoch 4/5

1064/1064 [=====] - 31s 29ms/step - loss: 0.2631 - accuracy: 0.9480 - val_loss: 0.3169 - val_accuracy: 0.9152

Epoch 5/5

1064/1064 [=====] - 32s 30ms/step - loss: 0.1843 - accuracy: 0.9618 - val_loss: 0.3107 - val_accuracy: 0.9179

Mejor época por pérdida de validación: 5



3.3 Evaluación: accuracy, matriz de confusión y reporte de métricas

Se evalúa una vez en las 3,080 consultas de test, manteniendo el orden original. La matriz tiene **clases reales en las filas y predicciones en las columnas**; se muestran los conteos y la normalización por fila (recall por clase en la diagonal). El reporte incluye precision, recall, F1 y support de las 77 clases, además de promedios macro y ponderados.

`zero_division=0` asigna cero si no hay predicciones para una clase.

```
In [16]: from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

def evaluate_predictions(true_labels, predicted_labels, names):
    # Fijar el orden de etiquetas mantiene alineadas las filas/columnas con
    label_ids = np.arange(len(names))
    matrix = confusion_matrix(true_labels, predicted_labels, labels=label_ids)
    report = classification_report(
        true_labels, predicted_labels, labels=label_ids,
        target_names=names, output_dict=True, zero_division=0,
    )
    per_class = pd.DataFrame([report[name] for name in names], index=names)
    per_class.index.name = "clase"
    per_class["support"] = per_class["support"].astype(int)
    # La diagonal son aciertos; suma por fila/columna permite separar falsos
    per_class["aciertos"] = np.diag(matrix)
    per_class["errores"] = matrix.sum(axis=1) - np.diag(matrix)
    per_class["falsos_positivos"] = matrix.sum(axis=0) - np.diag(matrix)
    return accuracy_score(true_labels, predicted_labels), matrix, report, per_class

# La evaluación conserva logits para calcular probabilidades softmax y etiqu
prediction_output = model.predict(test_dataset, verbose=1)
test_logits = prediction_output["logits"]
test_probabilities = tf.nn.softmax(test_logits, axis=-1).numpy()
predicted_labels = test_logits.argmax(axis=1)
assert len(predicted_labels) == len(evaluation_df)
test_accuracy, confusion, metrics_report, per_class_metrics = evaluate_predictions(
    test_labels, predicted_labels, class_names,
)
print(f"Accuracy: {test_accuracy:.4f} ({test_accuracy:.2%})")
```

```

print(classification_report(
    test_labels, predicted_labels, labels=np.arange(len(class_names)),
    target_names=class_names, digits=4, zero_division=0,
))
display(pd.DataFrame({
    "macro avg": metrics_report["macro avg"],
    "weighted avg": metrics_report["weighted avg"],
}).T)

confusion_df = pd.DataFrame(confusion, index=class_names, columns=class_names)
normalized_confusion = confusion / np.maximum(confusion.sum(axis=1, keepdims=True), 1)

for title, matrix, colorbar_label in [
    ("Matriz de confusión: conteos", confusion_df, "Consultas"),
    ("Matriz de confusión: normalización por clase real", normalized_confusion_df):
    fig, axis = plt.subplots(figsize=(24, 22))
    sns.heatmap(matrix, ax=axis, cmap="Blues", xticklabels=class_names,
                yticklabels=class_names, cbar_kws={"label": colorbar_label})
    axis.set(title=title, xlabel="Clase predicha", ylabel="Clase real")
    axis.tick_params(axis="both", labelsize=7)
    plt.xticks(rotation=90)
    plt.yticks(rotation=0)
    plt.tight_layout()
    plt.show()

per_class_metrics.to_csv(OUTPUT_DIR / "classification_report.csv")
confusion_df.to_csv(OUTPUT_DIR / "confusion_matrix.csv")

```

385/385 [=====] - 4s 6ms/step

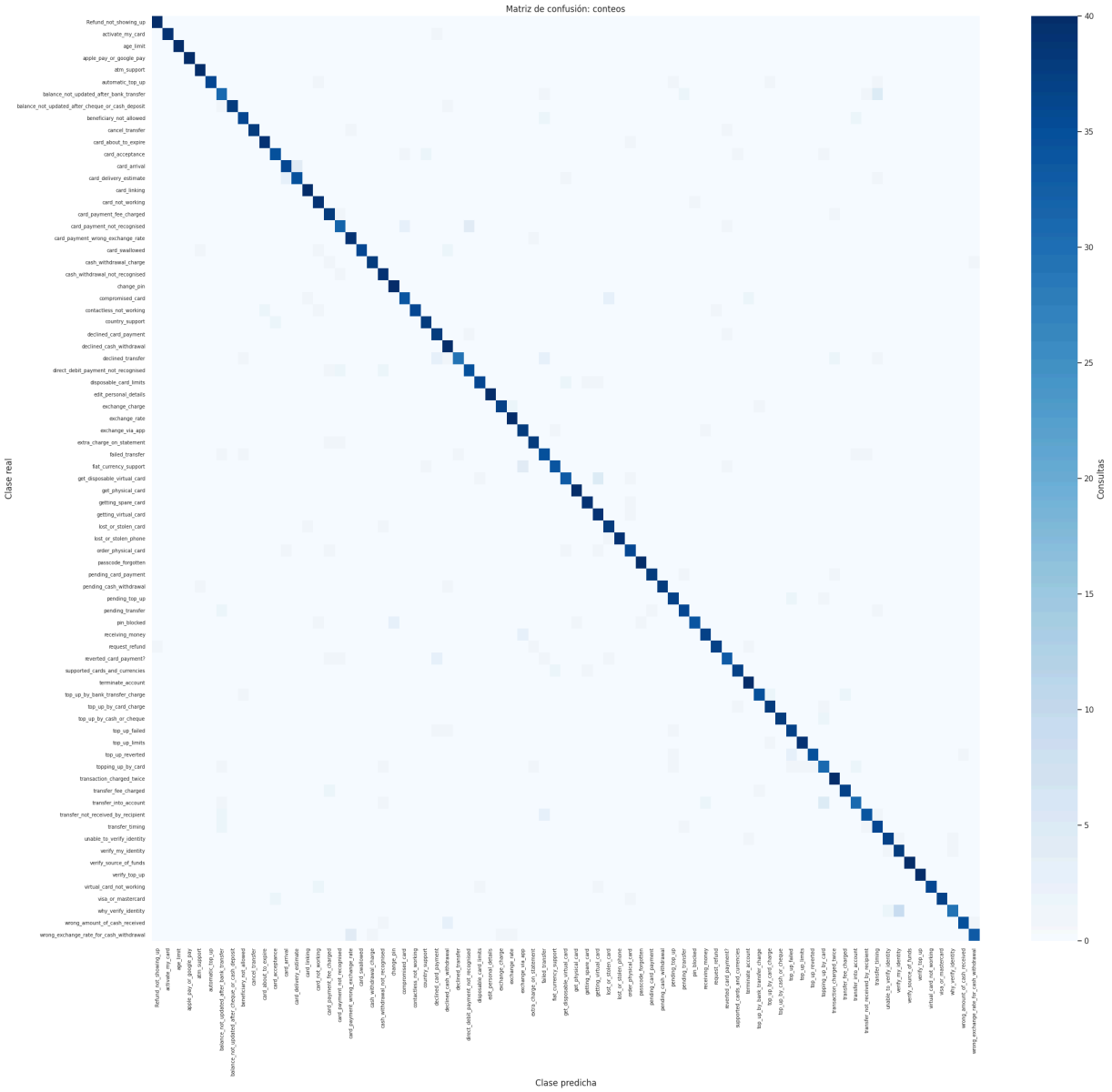
Accuracy: 0.9218 (92.18%)

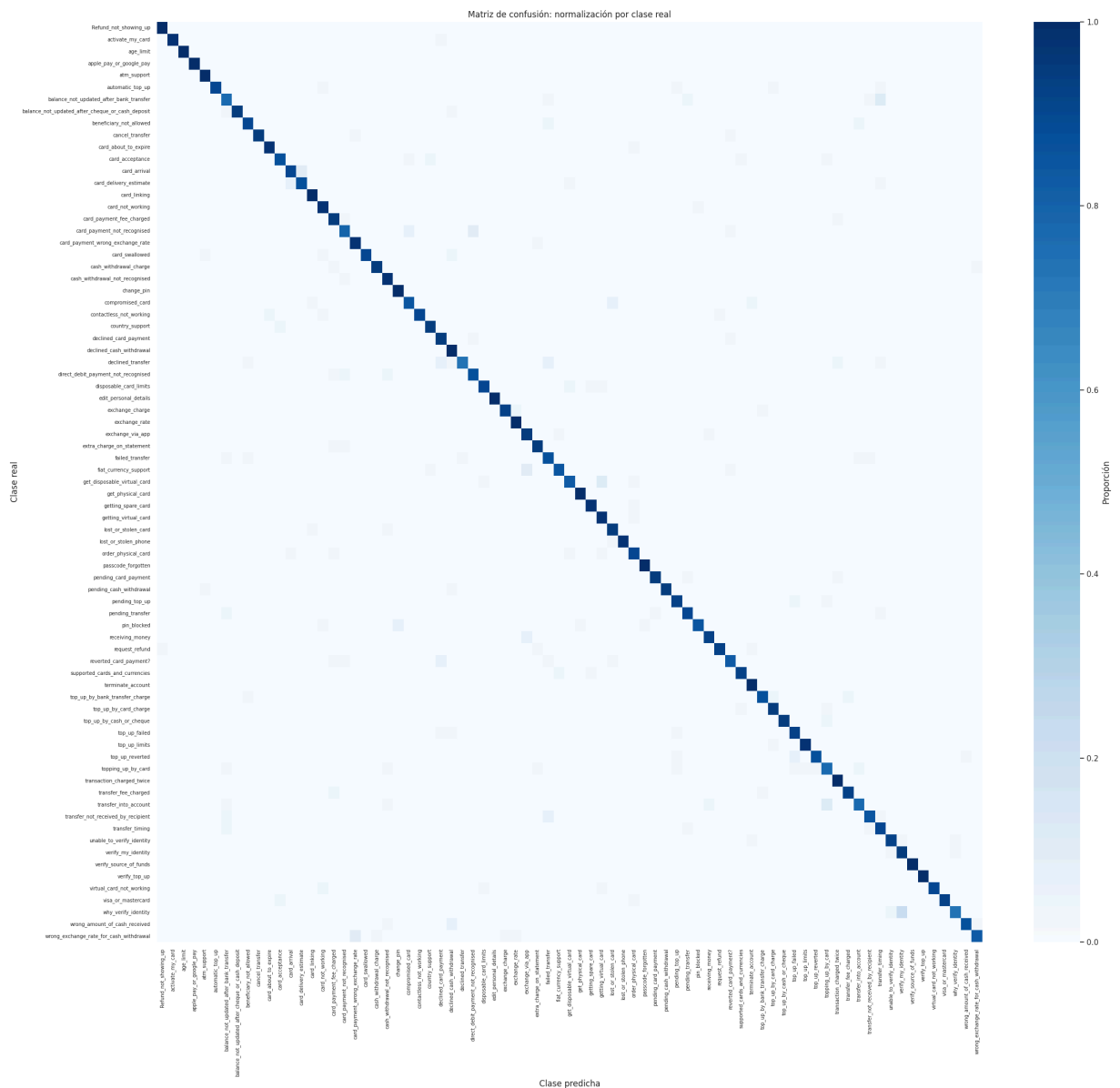
			precision	recall	f1-sco
re	support				
		Refund_not_showing_up	0.9756	1.0000	0.98
77	40				
		activate_my_card	1.0000	0.9750	0.98
73	40				
		age_limit	1.0000	1.0000	1.00
00	40				
		apple_pay_or_google_pay	1.0000	1.0000	1.00
00	40				
		atm_support	0.9524	1.0000	0.97
56	40				
		automatic_top_up	1.0000	0.9000	0.94
74	40				
		balance_not_updated_after_bank_transfer	0.7561	0.7750	0.76
54	40				
		balance_not_updated_after_cheque_or_cash_deposit	1.0000	0.9500	0.97
44	40				
		beneficiary_not_allowed	0.9231	0.9000	0.91
14	40				
		cancel_transfer	1.0000	0.9500	0.97
44	40				
		card_about_to_expire	0.9512	0.9750	0.96
30	40				
		card_acceptance	0.8974	0.8750	0.88
61	40				
		card_arrival	0.9000	0.9000	0.90
00	40				
		card_delivery_estimate	0.8974	0.8750	0.88
61	40				
		card_linking	0.9524	1.0000	0.97
56	40				
		card_not_working	0.8667	0.9750	0.91
76	40				
		card_payment_fee_charged	0.8444	0.9500	0.89
41	40				
		card_payment_not_recognised	0.8421	0.8000	0.82
05	40				
		card_payment_wrong_exchange_rate	0.8864	0.9750	0.92
86	40				
		card_swallowed	1.0000	0.9000	0.94
74	40				
		cash_withdrawal_charge	0.9744	0.9500	0.96
20	40				
		cash_withdrawal_not_recognised	0.8667	0.9750	0.91
76	40				
		change_pin	0.9302	1.0000	0.96
39	40				
		compromised_card	0.8947	0.8500	0.87
18	40				
		contactless_not_working	1.0000	0.9000	0.94
74	40				
		country_support	0.9268	0.9500	0.93

83	40			
		declined_card_payment	0.8261	0.9500
37	40			0.88
		declined_cash_withdrawal	0.8163	1.0000
89	40			0.89
		declined_transfer	0.9667	0.7250
86	40			0.82
		direct_debit_payment_not_recognised	0.8750	0.8750
50	40			0.87
		disposable_card_limits	0.9474	0.9000
31	40			0.92
		edit_personal_details	1.0000	1.0000
00	40			1.00
		exchange_charge	0.9737	0.9250
87	40			0.94
		exchange_rate	0.9302	1.0000
39	40			0.96
		exchange_via_app	0.8444	0.9500
41	40			0.89
		extra_charge_on_statement	0.9268	0.9500
83	40			0.93
		failed_transfer	0.7778	0.8750
35	40			0.82
		fiat_currency_support	0.9189	0.8500
31	40			0.88
		get_disposable_virtual_card	0.8919	0.8250
71	40			0.85
		get_physical_card	0.9756	1.0000
77	40			0.98
		getting_spare_card	0.9512	0.9750
30	40			0.96
		getting_virtual_card	0.8298	0.9750
66	40			0.89
		lost_or_stolen_card	0.8837	0.9500
57	40			0.91
		lost_or_stolen_phone	1.0000	0.9750
73	40			0.98
		order_physical_card	0.8780	0.9000
89	40			0.88
		passcode_forgotten	0.9756	1.0000
77	40			0.98
		pending_card_payment	0.9744	0.9500
20	40			0.96
		pending_cash_withdrawal	1.0000	0.9500
44	40			0.97
		pending_top_up	0.9024	0.9250
36	40			0.91
		pending_transfer	0.9000	0.9000
00	40			0.90
		pin_blocked	0.9714	0.8500
67	40			0.90
		receiving_money	0.9250	0.9250
50	40			0.92
		request_refund	0.9737	0.9250
87	40			0.94
		reverted_card_payment?	0.8919	0.8250
				0.85

71	40	supported_cards_and_currencies	0.9487	0.9250	0.93
67	40	terminate_account	0.9091	1.0000	0.95
24	40	top_up_by_bank_transfer_charge	0.9459	0.8750	0.90
91	40	top_up_by_card_charge	0.9048	0.9500	0.92
68	40	top_up_by_cash_or_cheque	1.0000	0.9500	0.97
44	40	top_up_failed	0.8605	0.9250	0.89
16	40	top_up_limits	0.9750	0.9750	0.97
50	40	top_up_reverted	0.9722	0.8750	0.92
11	40	topping_up_by_card	0.7561	0.7750	0.76
54	40	transaction_charged_twice	0.9091	1.0000	0.95
24	40	transfer_fee_charged	0.9487	0.9250	0.93
67	40	transfer_into_account	0.8611	0.7750	0.81
58	40	transfer_not_received_by_recipient	0.9189	0.8500	0.88
31	40	transfer_timing	0.7826	0.9000	0.83
72	40	unable_to_verify_identity	0.9250	0.9250	0.92
50	40	verify_my_identity	0.7917	0.9500	0.86
36	40	verify_source_of_funds	1.0000	1.0000	1.00
00	40	verify_top_up	1.0000	1.0000	1.00
00	40	virtual_card_not_working	1.0000	0.9000	0.94
74	40	visa_or_mastercard	1.0000	0.9250	0.96
10	40	why_verify_identity	0.9355	0.7250	0.81
69	40	wrong_amount_of_cash_received	0.9722	0.8750	0.92
11	40	wrong_exchange_rate_for_cash_withdrawal	0.9429	0.8250	0.88
00	40				
		accuracy			0.92
18	3080	macro avg	0.9250	0.9218	0.92
17	3080	weighted avg	0.9250	0.9218	0.92
17	3080				

	precision	recall	f1-score	support
macro avg	0.925014	0.921753	0.921666	3080.0
weighted avg	0.925014	0.921753	0.921666	3080.0





3.4 Siete clases mejor clasificadas y siete con más errores

Las mejores se ordenan por **F1 descendente**, con recall y nombre como desempate. Las problemáticas se ordenan por **número de consultas reales de la clase clasificadas incorrectamente** (falsos negativos), con F1 ascendente y nombre como desempate. Se muestran también falsos positivos: una clase puede recibir muchas consultas ajenas aunque tenga pocos falsos negativos.

En test cada clase tiene 40 consultas, por lo que comparar errores absolutos es equivalente a comparar recall. Los rankings describen esta ejecución y pueden cambiar con la semilla, el entrenamiento y las muestras.

```
In [17]: ranking = per_class_metrics.reset_index()
best_seven = ranking.sort_values(
```

```

["f1-score", "recall", "class"], ascending=[False, False, True],
).head(7)
worst_seven = ranking.sort_values(
    ["errores", "f1-score", "class"], ascending=[False, True, True],
).head(7)
print("Siete clases mejor clasificadas (F1):")
display(best_seven.round(4))
print("Siete clases con más errores (falsos negativos):")
display(worst_seven.round(4))

fig, axes = plt.subplots(1, 2, figsize=(16, 5))
sns.barplot(data=best_seven, x="f1-score", y="class", color="seagreen", ax=axes[0].set(title="Mejores clases por F1", xlim=(0, 1)))
sns.barplot(data=worst_seven, x="errores", y="class", color="firebrick", ax=axes[1].set_title("Más errores por clase real"))
plt.tight_layout()
plt.show()

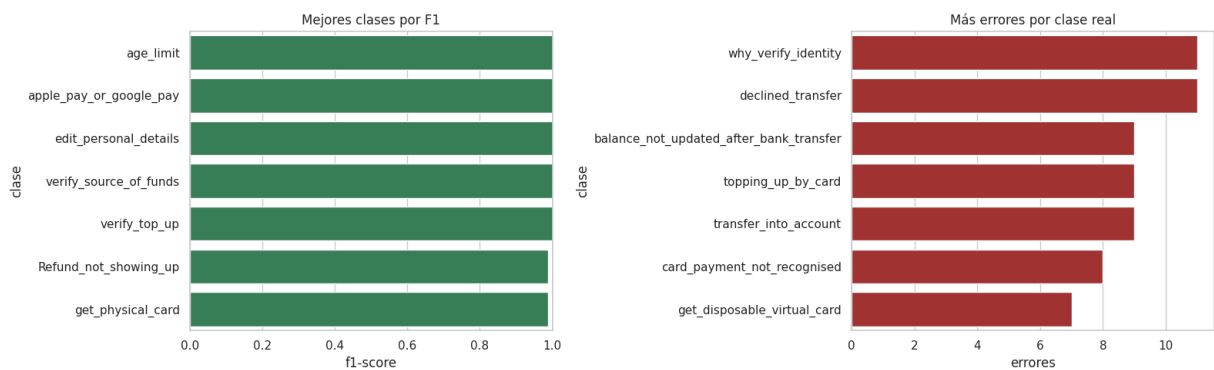
```

Siete clases mejor clasificadas (F1):

	clase	precision	recall	f1-score	support	aciertos	errores	falsos_pos
2	age_limit	1.0000	1.0	1.0000	40	40	0	
3	apple_pay_or_google_pay	1.0000	1.0	1.0000	40	40	0	
31	edit_personal_details	1.0000	1.0	1.0000	40	40	0	
70	verify_source_of_funds	1.0000	1.0	1.0000	40	40	0	
71	verify_top_up	1.0000	1.0	1.0000	40	40	0	
0	Refund_not_showing_up	0.9756	1.0	0.9877	40	40	0	
39	get_physical_card	0.9756	1.0	0.9877	40	40	0	

Siete clases con más errores (falsos negativos):

	clase	precision	recall	f1-score	support	aciertos	errores
74	why_verify_identity	0.9355	0.725	0.8169	40	29	
28	declined_transfer	0.9667	0.725	0.8286	40	29	
6	balance_not_updated_after_bank_transfer	0.7561	0.775	0.7654	40	31	
62	topping_up_by_card	0.7561	0.775	0.7654	40	31	
65	transfer_into_account	0.8611	0.775	0.8158	40	31	
17	card_payment_not_recognised	0.8421	0.800	0.8205	40	32	
38	get_disposable_virtual_card	0.8919	0.825	0.8571	40	33	



3.5 Análisis de clases problemáticas

Se inspeccionan las confusiones más frecuentes y hasta tres errores por clase, junto con el número de ejemplos usados para ajustar el modelo. Esto permite evaluar hipótesis:

- **Solapamiento semántico:** intenciones de tarjeta, transferencia, comisiones o estados pendientes pueden compartir vocabulario. Leer el objeto y el estado de cada operación en los errores, no solo el nombre de la clase.
- **Escasez o poca diversidad:** menos ejemplos de entrenamiento pueden limitar la generalización; los pesos de clase no generan nuevas formas de expresar una intención.
- **Ambigüedad y negaciones:** consultas cortas, contracciones o varios problemas en un mismo texto pueden requerir aclaraciones.
- **Truncamiento:** comparar la proporción de textos largos entre errores y aciertos para detectar pérdida potencial de información.
- **Optimización:** revisar las curvas train/validation; divergencia puede sugerir sobreajuste y bajo rendimiento en ambas puede sugerir ajuste insuficiente. Las pérdidas no son directamente comparables si se usan pesos de clase solo en train.

Los ejemplos y asociaciones son evidencia descriptiva, no prueba de causalidad. Las mejoras se seleccionan en validación; después de inspeccionar errores de test, las hipótesis deben confirmarse con otro conjunto independiente.

```
In [ ]: # Guardar errores como filas completas permite inspeccionar qué etiquetas se
error_analysis = evaluation_df[["text", "category"]].copy()
error_analysis["predicted_category"] = [id2label[int(label)] for label in pr
error_analysis["correct"] = test_labels == predicted_labels
error_analysis["max_probability"] = test_probabilities.max(axis=1)
error_analysis["token_length"] = test_token_lengths
error_analysis["truncated"] = test_token_lengths > MAX_LENGTH
classification_errors = error_analysis.loc[~error_analysis["correct"]].copy()
fit_category_support = fit_df["category"].value_counts()

# La confusión es dirigida: clase real A -> predicción B no equivale necesar
confusion_pairs = classification_errors.groupby(
    ["category", "predicted_category"],
).size().reset_index(name="errores").sort_values("errores", ascending=False)
print("Principales confusiones dirigidas (clase real -> predicción):")
```

```

display(confusion_pairs.head(15))
print("Truncamiento por resultado (media = proporción):")
display(error_analysis.groupby("correct")["truncated"].agg(["count", "mean"])

diagnostic_rows = []
for category in worst_seven["clase"]:
    category_errors = classification_errors.loc[classification_errors["category"] == category]
    counts = category_errors["predicted_category"].value_counts()
    main_confusion = counts.index[0] if len(counts) else "Sin errores"
    main_confusion_count = int(counts.iloc[0]) if len(counts) else 0
    diagnostic_rows.append({
        "clase": category,
        "train_usado": int(fit_category_support[category]),
        "F1": float(per_class_metrics.loc[category, "f1-score"]),
        "errores": len(category_errors),
        "confusión_principal": main_confusion,
        "errores_en_esa_confusión": main_confusion_count,
        "errores_truncados": int(category_errors["truncated"].sum()),
    })
notes = []
if len(counts):
    notes.append(
        f"{main_confusion_count} de {len(category_errors)} errores se dieron por truncamiento. "
        "revisar si las consultas distinguen el objeto y el estado de la clase"
    )
if fit_category_support[category] < fit_category_support.median():
    notes.append("Tiene menos ejemplos de ajuste que la mediana por clase")
if category_errors["truncated"].any():
    notes.append("Hay errores en textos truncados; comparar con sus versiones completas")
if not notes:
    notes.append("No hay errores en esta clase; su aparición se debe a errores de truncamiento")
display(Markdown(f"**{category}**\n\n" + "\n\n".join(notes)))
display(category_errors.sort_values("max_probability", ascending=False)[
    ["text", "predicted_category", "max_probability", "token_length", "tokens"].head(3))

diagnostics = pd.DataFrame(diagnostic_rows)
display(diagnostics.round(4))
error_analysis.to_csv(OUTPUT_DIR / "test_predictions.csv", index=False)
diagnostics.to_csv(OUTPUT_DIR / "problematic_classes.csv", index=False)

```

Principales confusiones dirigidas (clase real -> predicción):

	category	predicted_category	errores
152	why_verify_identity	verify_my_identity	9
8	balance_not_updated_after_bank_transfer	transfer_timing	5
73	get_disposable_virtual_card	getting_virtual_card	5
134	transfer_into_account	topping_up_by_card	5
28	card_payment_not_recognised	direct_debit_payment_not_recognised	4
156	wrong_exchange_rate_for_cash_withdrawal	card_payment_wrong_exchange_rate	4
20	card_arrival	card_delivery_estimate	4
70	fiat_currency_support	exchange_via_app	4
94	pin_blocked	change_pin	3
119	top_up_reverted	top_up_failed	3
38	compromised_card	lost_or_stolen_card	3
27	card_payment_not_recognised	compromised_card	3
47	declined_transfer	declined_card_payment	3
49	declined_transfer	failed_transfer	3
21	card_delivery_estimate	card_arrival	3

Truncamiento por resultado (media = proporción):

	count	mean
correct		
False	241	0.012448
True	2839	0.004931

why_verify_identity

9 de 11 errores se dirigen a `verify_my_identity`; revisar si las consultas distinguen el objeto y el estado de la operación.

Tiene menos ejemplos de ajuste que la mediana por clase: la cobertura lingüística es una hipótesis a revisar.

	text	predicted_category	max_probability	token_length	truncated
1193	Do I need to verify my identity?	verify_my_identity	0.947999	10	False
1192	What other methods are there to verify my iden...	verify_my_identity	0.935797	12	False
1187	I won't verify my identity.	unable_to_verify_identity	0.932108	9	False

declined_transfer

3 de 11 errores se dirigen a `declined_card_payment` ; revisar si las consultas distinguen el objeto y el estado de la operación.

	text	predicted_category	max_probability	token_length	truncated
1730	The card got declined twice when I tried to us...	transaction_charged_twice	0.927424	19	False
1726	How can I fix my card, it got declined twice.	transaction_charged_twice	0.927405	14	False
1736	Why was I unable to do a transfer?	failed_transfer	0.920621	11	False

balance_not_updated_after_bank_transfer

5 de 9 errores se dirigen a `transfer_timing` ; revisar si las consultas distinguen el objeto y el estado de la operación.

	text	predicted_category	max_probability	token_length	truncated
2711	My transfer is pending.	pending_transfer	0.958249	7	False
2701	I transferred some money but it is yet to arrive.	transfer_not_received_by_recipient	0.893758	13	False
2687	When will my transfer be available in my account.	transfer_timing	0.888743	12	False

topping_up_by_card

2 de 9 errores se dirigen a `transfer_into_account` ; revisar si las consultas distinguen el objeto y el estado de la operación.

Tiene menos ejemplos de ajuste que la mediana por clase: la cobertura lingüística es una hipótesis a revisar.

	text	predicted_category	max_probability	token_length	truncated
1340	Is there a way to transfer funds directly from...	transfer_into_account	0.908913	14	False
1347	Cannot access my top up.	top_up_failed	0.819723	9	False
1351	Why is my money gone right when I attempted to...	top_up_reverted	0.663335	14	False

transfer_into_account

5 de 9 errores se dirigen a `topping_up_by_card` ; revisar si las consultas distinguen el objeto y el estado de la operación.

Tiene menos ejemplos de ajuste que la mediana por clase: la cobertura lingüística es una hipótesis a revisar.

	text	predicted_category	max_probability	token_length	trur
2338	How do I top up my card?	topping_up_by_card	0.952153	10	
2323	I don't understand how to top up my account, c...	topping_up_by_card	0.923946	20	
2324	I don't understand the money transfer process.	balance_not_updated_after_bank_transfer	0.792469	11	

card_payment_not_recognised

4 de 8 errores se dirigen a `direct_debit_payment_not_recognised`; revisar si las consultas distinguen el objeto y el estado de la operación.

Hay errores en textos truncados; comparar con sus versiones completas antes de atribuirlos al modelo.

	text	predicted_category	max_probability	token_length	truncated
1103	What should I do to get transactions off of my...	compromised_card	0.949849	36	False
1112	Somebody used my card to make a purchase	compromised_card	0.902071	11	False
1096	Can you freeze my account? I just saw there a...	compromised_card	0.854624	31	False

get_disposable_virtual_card

5 de 7 errores se dirigen a `getting_virtual_card`; revisar si las consultas distinguen el objeto y el estado de la operación.

Tiene menos ejemplos de ajuste que la mediana por clase: la cobertura lingüística es una hipótesis a revisar.

	text	predicted_category	max_probability	token_length	truncated
2637	how many transactions can i make with a dispos...	disposable_card_limits	0.973548	12	False
2622	how does a virtual card work	getting_virtual_card	0.921508	8	False
2606	I would like a temporary virtual card	getting_virtual_card	0.898205	9	False

	clase	train_usado	F1	errores	confu:
0	why_verify_identity	103	0.8169	11	ver
1	declined_transfer	113	0.8286	11	declined_
2	balance_not_updated_after_bank_transfer	145	0.7654	9	
3	topping_up_by_card	88	0.7654	9	transfe
4	transfer_into_account	96	0.8158	9	toppi
5	card_payment_not_recognised	143	0.8205	8	direct_debit_payment_
6	get_disposable_virtual_card	82	0.8571	7	getti

3.6 Conclusiones del uso de DistilRoBERTa

La siguiente celda redacta el resumen cuantitativo **después del entrenamiento y la evaluación**, identificando las clases concretas de esta ejecución. No se presume una accuracy ni una lista de clases antes de obtener las predicciones.

DistilRoBERTa permite transferir conocimiento lingüístico y ajustar representaciones contextuales, una ventaja potencial cuando varias intenciones comparten palabras. Sin embargo, para demostrar una mejora se necesita comparar con un baseline bajo el mismo protocolo y repetir el entrenamiento con distintas semillas.

Para clasificar consultas bancarias en línea:

- BANKING77 está en inglés y cubre 77 intenciones; no prueba desempeño en español ni en consultas fuera del dominio.
- Test tiene 40 consultas por clase. Una sola equivocación modifica el recall de una clase en 2.5 puntos porcentuales; no sobreinterpretar diferencias pequeñas.
- Medir latencia, memoria y throughput en el hardware de despliegue. El menor tamaño respecto a RoBERTa no garantiza una latencia aceptable sin medirla.
- Las probabilidades softmax no son confianza calibrada. Calibrar y elegir umbrales en validación para derivar consultas ambiguas o desconocidas a asistencia humana.

- Proteger datos personales y monitorizar cambios de vocabulario, confusiones y rendimiento por clase. Priorizar ejemplos diversos y revisar etiquetas en las clases problemáticas antes de aplicar sobremuestreo.
- Conservar juntos el modelo, el tokenizer, `label2id`, `id2label` y la longitud máxima. El entrenamiento guarda estos artefactos y los reportes en `distilroberta_banking77/`.

La función final devuelve las tres intenciones con mayor probabilidad para consultas nuevas, sin entrenar otra vez. Esta demostración no constituye un servicio de producción.

```
In [19]: total_errors = int((test_labels != predicted_labels).sum())
best_names = ", ".join(best_seven["clase"])
worst_names = ", ".join(worst_seven["clase"])
if len(confusion_pairs):
    main_pair = confusion_pairs.iloc[0]
    confusion_summary = (
        f"La confusión dirigida más frecuente fue `{main_pair['category']}`\n"
        f"`{main_pair['predicted_category']}` ({int(main_pair['errores'])} c\n"
        "Los ejemplos anteriores permiten revisar el solapamiento semántico,\n"
    )
else:
    confusion_summary = "No hubo errores en test en esta ejecución; aun así,\n"

summary = (
    f"### Resumen de esta ejecución\n\n"
    f"El ajuste de DistilRoBERTa seleccionó la época **{best_epoch}** usando\n"
    f"En las **{len(test_labels):,}** consultas de test obtuvo **accuracy {t\n"
    f"**F1-macro {metrics_report['macro avg']['f1-score']:.4f}** y "\n"
    f"**F1 ponderado {metrics_report['weighted avg']['f1-score']:.4f}**;\n"
    f"se clasificaron incorrectamente **{total_errors}** consultas.\n\n"
    f"**Mejores siete por F1:** {best_names}.\n\n"
    f"**Siete con más falsos negativos (incluyendo empates):** {worst_names}\n"
    f"{confusion_summary}\n\n"
    f"Se truncó el **{np.mean(test_token_lengths > MAX_LENGTH):.2%}** de tes\n"
    f"una longitud máxima de **{MAX_LENGTH}** tokens elegida solo con entren\n"
    "Las métricas describen BANKING77, no consultas reales fuera de esta dis\n"
    "Antes del despliegue, comparar con un baseline, comprobar latencia y ca\n"
    "y validar de manera independiente las mejoras propuestas tras inspeccio\n"
)
display(Markdown(summary))
(OUTPUT_DIR / "conclusions.txt").write_text(summary, encoding="utf-8")

def classify_queries(texts, top_k=3):
    if isinstance(texts, str):
        texts = [texts]
    if not texts or any(not isinstance(text, str) or not text.strip() for te
        raise ValueError("Proporciona una consulta o una lista de consultas")
    if not isinstance(top_k, int) or not 1 <= top_k <= len(class_names):
        raise ValueError(f"top_k debe estar entre 1 y {len(class_names)}.")
    encoding = dict(tokenizer(
        texts, padding=True, truncation=True, max_length=MAX_LENGTH, return_
    ))
```

```

logits = model(encoding, training=False).logits
probabilities = tf.nn.softmax(logits, axis=-1).numpy()
rows = []
for text, scores in zip(texts, probabilities):
    top_ids = np.argsort(-scores)[:top_k]
    for position, label in enumerate(top_ids, start=1):
        rows.append({
            "consulta": text, "posición": position,
            "intención": model.config.id2label[int(label)],
            "probabilidad_softmax": float(scores[label]),
        })
return pd.DataFrame(rows)

display(classify_queries(["My card is not working", "How do I activate my ca

```

Resumen de esta ejecución

El ajuste de DistilRoBERTa seleccionó la época **5** usando la pérdida de validación. En las **3,080** consultas de test obtuvo **accuracy 92.18%**, **F1-macro 0.9217** y **F1 ponderado 0.9217**; se clasificaron incorrectamente **241** consultas.

Mejores siete por F1: age_limit, apple_pay_or_google_pay, edit_personal_details, verify_source_of_funds, verify_top_up, Refund_not_showing_up, get_physical_card.

Siete con más falsos negativos (incluyendo empates): why_verify_identity, declined_transfer, balance_not_updated_after_bank_transfer, topping_up_by_card, transfer_into_account, card_payment_not_recognised, get_disposable_virtual_card.

La confusión dirigida más frecuente fue `why_verify_identity` hacia `verify_my_identity` (9 consultas). Los ejemplos anteriores permiten revisar el solapamiento semántico, sin demostrar causalidad.

Se truncó el **0.55%** de test con una longitud máxima de **51** tokens elegida solo con entrenamiento. Las métricas describen BANKING77, no consultas reales fuera de esta distribución. Antes del despliegue, comparar con un baseline, comprobar latencia y calibración, y validar de manera independiente las mejoras propuestas tras inspeccionar test.

	consulta	posición	intención	probabilidad_softmax
0	My card is not working	1	card_not_working	0.969918
1	My card is not working	2	declined_card_payment	0.007042
2	My card is not working	3	lost_or_stolen_card	0.001475
3	How do I activate my card?	1	activate_my_card	0.987610
4	How do I activate my card?	2	card_linking	0.000651
5	How do I activate my card?	3	pin_blocked	0.000567

4. Interpretación de errores con Falcon-7B-Instruct

Modelo: `tiiaue/falcon-7b-instruct` . Se genera una **hipótesis textual post hoc**, no una explicación causal de los mecanismos de DistilRoBERTa. El LLM no tiene acceso a sus activaciones ni al proceso que produjo la predicción y puede racionalizar un error.

4.1 Prompt, estructura y parámetros de generación

El prompt separa instrucciones y datos JSON: consulta original, clase predicha y etiqueta real. La etiqueta real se incluye para auditar una clasificación ya evaluada, no para inferencia en producción. Se exige JSON con una razón de un catálogo, un fragmento literal de la consulta y una explicación de hasta **cuatro oraciones completas**. Primero se buscan razones concretas; ambigüedad requiere dos lecturas respaldadas por el texto y la falta de evidencia es la última opción. Se prohíben hechos bancarios inventados y supuestas causas internas.

Configuración conservadora inicial: decodificación **greedy** (`do_sample=False` , temperatura efectiva 0, sin pasar `temperature=0` a la API) y `max_new_tokens=160` . El prompt incluye un ejemplo JSON breve para reducir respuestas narrativas; el límite de tokens no garantiza el formato, por lo que se validan estructura, brevedad y evidencia. Una temperatura baja reduce variación, pero **no elimina alucinaciones**.

Antes de los 20 errores se comparan greedy/160 tokens, muestreo con temperatura 0.2/160 tokens y greedy/192 tokens en tres casos sintéticos independientes de test. La revisión humana comprueba fundamentación, contraste de etiquetas y coherencia; no se declara calibración empírica sin esa revisión.

4.2 Selección reproducible de 20 errores

Se toma una muestra aleatoria sin reemplazo (`random_state=42`) de errores reales de test. Cada consulta debe tener etiqueta real distinta de la predicción; si hay menos de 20, se detiene sin inventar ni repetir muestras. La selección y los resultados se guardan para auditoría.

Recursos: En CUDA, Falcon usa cuantización de 8 bits con bitsandbytes, reduciendo el espacio de los pesos a aproximadamente 7 GB, más memoria de ejecución. En Linux, selecciona el kernel **Python (Transformers - PyTorch GPU)**; MPS conserva FP16 y CPU FP32 (~28 GB solo en pesos). Tras instalar o actualizar dependencias, reinicia ese kernel antes de ejecutar esta sección. Guarda primero las predicciones de DistilRoBERTa. Esta sección recupera los errores desde

`distilroberta_banking77/test_predictions.csv` , por lo que puede ejecutarse

tras cambiar de kernel. Ejecuta un modelo GPU a la vez para liberar memoria; no se eliminan automáticamente modelos ni variables existentes.

```
In [3]: import json
        from pathlib import Path

import pandas as pd
from IPython.display import Markdown, display

FALCON_MODEL_NAME = "tiiuae/falcon-7b-instruct"
FALCON_SEED = 42
FALCON_OUTPUT_DIR = Path("falcon_banking77_analysis")
FALCON_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
REASON_DESCRIPTIONS = {
    "semantic_overlap": "Solapamiento semántico entre conceptos de las etiquetas",
    "negation_or_status": "Negación o estado de la operación en la consulta",
    "unsupported_prediction": "La etiqueta predicha no tiene apoyo visible en la consulta",
    "ambiguity": "La consulta admite más de una interpretación",
    "insufficient_evidence": "La consulta no aporta información suficiente",
}

GENERATION_CONFIGS = {"greedy_160": {"do_sample": False, "max_new_tokens": 128}}
SELECTED_CONFIG = "greedy_160"

def select_falcon_errors(predictions, sample_size=20):
    # El muestreo sin reemplazo y la semilla fija hacen reproducible la revisión
    required = {"text", "category", "predicted_category"}
    if not required.issubset(predictions.columns):
        raise ValueError(f"Faltan columnas: {required - set(predictions.columns)}")
    frame = predictions.copy()
    if frame[list(required)].isna().any().any():
        raise ValueError("Las consultas y etiquetas no pueden contener valores faltantes")
    frame["test_row"] = frame.index
    errors = frame.loc[frame["category"] != frame["predicted_category"]]
    # Fallar si faltan casos evita duplicar errores o crear ejemplos sintéticos
    if len(errors) < sample_size:
        raise ValueError(f"Hay {len(errors)} errores; se necesitan {sample_size - len(errors)} más")
    return errors.sample(n=sample_size, random_state=FALCON_SEED).reset_index(drop=True)

if "error_analysis" in globals():
    falcon_prediction_frame = error_analysis
else:
    predictions_path = Path("distilroberta_banking77/test_predictions.csv")
    if not predictions_path.exists():
        raise FileNotFoundError("Ejecuta primero la evaluación y el análisis")
    falcon_prediction_frame = pd.read_csv(predictions_path)

falcon_samples = select_falcon_errors(falcon_prediction_frame)
assert len(falcon_samples) == 20 and falcon_samples["test_row"].is_unique
falcon_samples.to_csv(FALCON_OUTPUT_DIR / "selected_errors.csv", index=False)
display(Markdown(falcon_samples[["test_row", "text", "category", "predicted_category"]].to_csv(index=False, header=False))))

def build_falcon_prompt(query, predicted_label, true_label):
```

```

payload = json.dumps({
    "consulta": str(query),
    "etiqueta_predicha": str(predicted_label),
    "etiqueta_referencia": str(true_label),
}, ensure_ascii=False)
reason_keys = ", ".join(REASON_DESCRIPTIONS)
return (
    "User: Analiza esta clasificación bancaria únicamente como hipótesis  

    "No afirmes conocer el mecanismo interno del clasificador ni invente  

    "Devuelve exactamente un objeto JSON con las claves string reason, e  

    f"reason debe ser una de estas claves: {reason_keys}. Analiza primer  

    "Prioriza una razón concreta: solapamiento de conceptos, negación o  

    "para la etiqueta predicha. No elijas ambiguity por defecto: úsala s  

    "que permitan al menos dos interpretaciones plausibles y explica cuánta  

    "La prioridad es proponer una razón y una posible justificación de p  

    "y la de referencia. No elijas insufficient_evidence solo porque no  

    "el significado de la consulta y las etiquetas, y formula una hipóte  

    "solo como última opción, cuando no puedas proponer ninguna explicac  

    "justification debe limitarse a indicar brevemente la falta de evide  

    "una frase larga fija. Puedes inferir una causa probable cuando el t  

    "pero exprésala como posibilidad (por ejemplo, 'podría deberse a...'  

    "evidence debe ser una cita literal breve si hay una; de lo contrari  

    "la razón y la hipótesis si el significado del texto permite formula  

    "justification debe ser específica para este caso, sin repetir la co  

    "que expliquen la razón, comparen las etiquetas y distingan indicios  

    "Para ambiguity, describe las dos interpretaciones concretas respald  

    "No afirmes causalidad ni determines si la predicción es válida o in  

    "No añadas texto fuera del JSON.\n"  

    "Ejemplo: {"reason\":\"semantic_overlap\",\"evidence\":\"card payme  

    "\"justification\":\"La frase «card payment» nombra un pago con tarj  

    "con un cobro como con una comisión vinculada al pago, por lo que la  

    "La consulta no aclara cuál de esos aspectos originó la confusión.\n"  

    "Input: " + payload + "\nAssistant: {"  

)

print(build_falcon_prompt(
    falcon_samples.loc[0, "text"],
    falcon_samples.loc[0, "predicted_category"],
    falcon_samples.loc[0, "category"],
))

```

test_row	text	category	predicted_c
0	368 How do I unblock my card using the app?	card_not_working	pin_
1	173 Where did this fee come from?	extra_charge_on_statement	card_payment_fee_
2	2701 I transferred some money but it is yet to arrive.	balance_not_updated_after_bank_transfer	transfer_not_received_by_u
3	2604 how do VR cards work	get_disposable_virtual_card	getting_virtu
4	2939 I will need to get a new card soon, how do I o...	card_about_to_expire	order_physic
5	2248 Is it possible for me to get money out in a di...	receiving_money	exchange_
6	2523 My non-physical card will not work	virtual_card_not_working	card_not_
7	2691 How long does a UK transfer take?	balance_not_updated_after_bank_transfer	transfe
8	246 There is an incoming payment into my account, ...	fiat_currency_support	reverted_card_pi
9	1381 How can I create many temporary cards daily?	disposable_card_limits	getting_spe

test_row	text	category	predicted_c
10	673 Where is the money I topped off with?	pending_top_up	topping_up_
11	2592 The see the rate applied to my transaction was...	wrong_exchange_rate_for_cash_withdrawal	card_payment_wrong_exchar
12	2711 My transfer is pending.	balance_not_updated_after_bank_transfer	pending_
13	2606 I would like a temporary virtual card	get_disposable_virtual_card	getting_virtu
14	1101 I think my statement has a fraudulent charge	card_payment_not_recognised	direct_debit_payment_not_rec
15	1756 My card is being declined online. Could you te...	declined_transfer	declined_card_
16	2341 What methods can I use to add money to my acco...	transfer_into_account	topping_up_
17	280 I need my card now!	card_delivery_estimate	card
18	945 Can I get a copy of the card by email?	getting_virtual_card	order_physic
19	1517 I was charged on my account that shouldn't be ...	direct_debit_payment_not_recognised	card_payment_fee_

User: Analiza esta clasificación bancaria únicamente como hipótesis textual. No afirmes conocer el mecanismo interno del clasificador ni inventes hechos del cliente. Devuelve exactamente un objeto JSON con las claves string reason, evidence y justification. reason debe ser una de estas claves: semantic_overlap, negation_or_status, unsupported_prediction, ambiguity, insufficient_evidence. Analiza primero la consulta y compara las dos etiquetas. Prioriza una razón concreta: solapamiento de conceptos, negación o estado, o falta de apoyo textual para la etiqueta predicha. No elijas ambiguity por defecto: úsala solo si hay palabras concretas que permitan al menos dos interpretaciones plausibles y explica cuáles son. La prioridad es proponer una razón y una posible justificación de por qué difieren la etiqueta predicha y la de referencia. No elijas insufficient_evidence solo porque no encuentres una cita literal: compara el significado de la consulta y las etiquetas, y formula una hipótesis razonable. Usa insufficient_evidence solo como última opción, cuando no puedas proponer ninguna explicación plausible. En ese caso, justification debe limitarse a indicar brevemente la falta de evidencia, sin repetir la consulta ni añadir una frase larga fija. Puedes inferir una causa probable cuando el texto y las etiquetas la hagan razonable, pero exprésala como posibilidad (por ejemplo, 'podría deberse a...' o 'es posible que...'), nunca como hecho. evidence debe ser una cita literal breve si hay una; de lo contrario, usa una cadena vacía, pero conserva la razón y la hipótesis si el significado del texto permite formularlas. justification debe ser específica para este caso, sin repetir la consulta: hasta cuatro oraciones breves que expliquen la razón, comparen las etiquetas y distingan indicios directos de inferencias posibles. Para ambiguity, describe las dos interpretaciones concretas respaldadas por el texto. No afirmes causalidad ni determines si la predicción es válida o inválida. No añadas texto fuera del JSON.

Ejemplo: {"reason": "semantic_overlap", "evidence": "card payment", "justification": "La frase «card payment» nombra un pago con tarjeta. Ese indicio puede asociarse tanto con un cobro como con una comisión vinculada al pago, por lo que las etiquetas podrían solaparse. La consulta no aclara cuál de esos aspectos originó la confusión."}

Input: {"consulta": "How do I unblock my card using the app?", "etiqueta_predicha": "pin_blocked", "etiqueta_referencia": "card_not_working"}

Assistant: {

```
In [4]: import os
import psutil
import truststore
import torch
from huggingface_hub import HfApi, constants as hf_constants
from transformers import (
    AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig, GenerationConfig,
    StoppingCriteria, StoppingCriteriaList, is_bitsandbytes_available, set_seed
)

truststore.inject_into_ssl()
os.environ["HF_HUB_DISABLE_XET"] = "1"
hf_constants.HF_HUB_DISABLE_XET = True

if torch.cuda.is_available():
    if not is_bitsandbytes_available():
        raise RuntimeError(
            "bitsandbytes no está disponible para este proceso de Transformers. "
            "Selecciona el kernel Python (Transformers - PyTorch GPU), instálalo "
            "y reinicia el kernel antes de continuar."
```

```

    )
    falcon_device = "cuda:0"
    falcon_dtype = torch.float16
    available_bytes, _ = torch.cuda.mem_get_info(0)
    required_gib = 10
    falcon_quantization = BitsAndBytesConfig(load_in_8bit=True)
elif torch.backends.mps.is_available():
    falcon_device = "mps"
    falcon_dtype = torch.float16
    available_bytes = psutil.virtual_memory().available
    required_gib = 18
    falcon_quantization = None
else:
    falcon_device = "cpu"
    falcon_dtype = torch.float32
    available_bytes = psutil.virtual_memory().available
    required_gib = 34
    falcon_quantization = None

available_gib = available_bytes / 1024 ** 3
if available_gib < required_gib:
    raise RuntimeError(
        f"Falcon en {falcon_device}: {available_gib:.1f} GiB libres, se reco
        "Las 20 muestras ya están guardadas. Cierra el kernel de DistilRoBER
        "en un kernel nuevo, o usa un equipo con más memoria."
    )

# Para resetear la memoria utilizada en los kernels de vscode: Abre la palet
# Busca Developer: Reload Window y ejecútalo.

FALCON_REVISION = HfApi().model_info(FALCON_MODEL_NAME).sha
quantization_status = "8-bit" if falcon_quantization is not None else "sin c
print(f"Falcon: {falcon_device}, {quantization_status}, {falcon_dtype}, revi
falcon_tokenizer = AutoTokenizer.from_pretrained(FALCON_MODEL_NAME, revision
falcon_tokenizer.pad_token = falcon_tokenizer.eos_token
falcon_model = AutoModelForCausalLM.from_pretrained(
    FALCON_MODEL_NAME, revision=FALCON_REVISION,
    torch_dtype=falcon_dtype, low_cpu_mem_usage=True,
    device_map={"": falcon_device}, trust_remote_code=False,
    quantization_config=falcon_quantization,
    use_safetensors=True, attn_implementation="eager",
)
falcon_model.eval()

def json_response_complete(response):
    # Detener solo al cerrar un objeto JSON parseable; esto valida formato,
    try:
        return isinstance(json.loads(response.strip()), dict)
    except (json.JSONDecodeError, TypeError):
        return False

class StopAtJsonEnd(StoppingCriteria):
    def __init__(self, tokenizer, prompt_length, response_prefix="{"):
        self.tokenizer = tokenizer

```

```

        self.prompt_length = prompt_length
        self.response_prefix = response_prefix

    def __call__(self, input_ids, scores, **kwargs):
        response = self.response_prefix + self.tokenizer.decode(
            input_ids[0, self.prompt_length:], skip_special_tokens=True,
        )
        return json_response_complete(response)

def explain_falcon_case(query, predicted_label, true_label, config_name):
    prompt = build_falcon_prompt(query, predicted_label, true_label)
    encoded = falcon_tokenizer(prompt, return_tensors="pt", truncation=False)
    # Al ser un modelo causal, la salida contiene prompt + continuación; por lo tanto
    prompt_length = encoded["input_ids"].shape[1]
    settings = GENERATION_CONFIGS[config_name].copy()
    context_limit = getattr(falcon_model.config, "max_position_embeddings", 1024)

    if prompt_length + settings["max_new_tokens"] > context_limit:
        raise ValueError("El prompt excede el contexto de Falcon; no se truncó")

    generation_config = GenerationConfig(
        **settings, num_beams=1, repetition_penalty=1.05,
        eos_token_id=falcon_tokenizer.eos_token_id,
        pad_token_id=falcon_tokenizer.pad_token_id,
    )
    set_seed(FALCON_SEED)
    encoded = {name: tensor.to(falcon_device) for name, tensor in encoded.items()}

    with torch.inference_mode():
        output = falcon_model.generate(
            **encoded, generation_config=generation_config,
            stopping_criteria=StoppingCriteriaList([StopAtJsonEnd(falcon_tokenizer)
            ])

        # El prefijo "{" ya forma parte de la instrucción de continuación y se reemplaza
        new_tokens = output[0, prompt_length:]
        raw_response = "{" + falcon_tokenizer.decode(new_tokens, skip_special_tokens=True)
        parsed, status = parse_falcon_response(raw_response, query)

    return {
        "raw_response": raw_response, "validation_status": status,
        "reason": parsed["reason"] if parsed else None,
        "reported_reason": parsed["reported_reason"] if parsed else None,
        "reason_normalized": parsed["reason_normalized"] if parsed else False,
        "evidence": parsed["evidence"] if parsed else None,
        "explanation": parsed["explanation"] if parsed else None,
        "generated_tokens": len(new_tokens),
        "token_limit_reached": len(new_tokens) >= settings["max_new_tokens"]
    }

```

Falcon: cuda:0, 8-bit, torch.float16, revisión 8782b5c5d8c9290412416618f36a133653e85285

```
/home/panchis/.venvs/transformers-torch-gpu/lib/python3.10/site-packages/transformers/tokenization_utils_base.py:1601: FutureWarning: `clean_up_tokenization_spaces` was not set. It will be set to `True` by default. This behavior will be deprecated in transformers v4.45, and will be then set to `False` by default. For more details check this issue: https://github.com/huggingface/transformers/issues/31884
```

```
warnings.warn(  
Loading checkpoint shards: 100%|██████████| 2/2 [00:02<00:00, 1.20s/it]
```

4.3 Configuración del prompt y la generación

Se usa decodificación greedy (`do_sample=False`), equivalente a temperatura 0: evita variaciones aleatorias al pedir razones comparables. El límite de 160 tokens nuevos deja espacio para el objeto JSON y una justificación más desarrollada, de hasta cuatro oraciones. El prompt prioriza razones textuales concretas; reserva la ambigüedad para cuando el texto respalda dos lecturas y usa la falta de evidencia como último recurso. Estos parámetros priorizan consistencia y detalle, no garantizan veracidad.

```
In [ ]: def explain_falcon_case(query, predicted_label, true_label, config_name=SELECTED_CONFIG_NAME):  
    prompt = build_falcon_prompt(query, predicted_label, true_label)  
    encoded = falcon_tokenizer(prompt, return_tensors="pt", truncation=False)  
    prompt_length = encoded["input_ids"].shape[1]  
    settings = GENERATION_CONFIGS[config_name].copy()  
    context_limit = getattr(falcon_model.config, "max_position_embeddings", 1024)  
    if prompt_length + settings["max_new_tokens"] > context_limit:  
        raise ValueError("El prompt excede el contexto de Falcon; no se truncó")  
    generation_config = GenerationConfig(  
        **settings, num_beams=1, repetition_penalty=1.05,  
        eos_token_id=falcon_tokenizer.eos_token_id,  
        pad_token_id=falcon_tokenizer.pad_token_id,  
    )  
    set_seed(FALCON_SEED)  
    encoded = {name: tensor.to(falcon_device) for name, tensor in encoded.items()}  
    with torch.inference_mode():  
        output = falcon_model.generate(**encoded, generation_config=generation_config)  
        new_tokens = output[0, prompt_length:]  
        raw_response = "{" + falcon_tokenizer.decode(new_tokens, skip_special_tokens=True)  
    try:  
        parsed = json.loads(raw_response)  
    except (json.JSONDecodeError, TypeError):  
        parsed = {}  
    if not isinstance(parsed, dict):  
        parsed = {}  
    reason = parsed.get("reason")  
    if not isinstance(reason, str) or reason not in REASON_DESCRIPTIONS:  
        reason = "insufficient_evidence"  
    evidence = parsed.get("evidence")  
    if not isinstance(evidence, str):  
        evidence = ""  
    evidence = evidence.strip()  
    if not evidence or evidence not in str(query):  
        evidence = ""
```

```

justification = parsed.get("justification")
if not isinstance(justification, str) or not justification.strip():
    reason = "insufficient_evidence"
    justification = "Evidencia insuficiente para determinar una causa co
else:
    justification = justification.strip()
return {
    "reason": reason,
    "evidence": evidence,
    "justification": justification,
    "raw_response": raw_response,
    "generated_tokens": len(new_tokens),
}

print(f"Configuración seleccionada: {SELECTED_CONFIG}; máximo {GENERATION_CO

```

Configuración seleccionada: greedy_160; máximo 160 tokens nuevos.

4.4 Justificaciones de las 20 clasificaciones incorrectas

Se genera una respuesta por cada error seleccionado. Para cada consulta se guardan la etiqueta real, la predicha, la razón sugerida por Falcon, una posible cita textual y una justificación específica de hasta cuatro oraciones. La prioridad es proponer por qué difieren las etiquetas y una justificación posible; se permiten inferencias razonables si se expresan como posibilidades, aunque no haya una cita literal. La ambigüedad requiere dos interpretaciones concretas; `insufficient_evidence` solo se usa como último recurso si no hay una explicación plausible. La respuesta original también se conserva para revisarla.

```

In [ ]: results = []
# La predicción y la etiqueta real se envían junto con el texto para analiza
for position, sample in enumerate(falcon_samples.to_dict(orient="records"),
    response = explain_falcon_case(
        sample["text"], sample["predicted_category"], sample["category"], SE
    )
# Conservar la consulta y sus etiquetas junto a la explicación facilita
results.append(**sample, "generation_config": SELECTED_CONFIG, **respon
current_results = pd.DataFrame(results)
# Actualizar ambos archivos en cada vuelta para no perder los casos comp
current_results.to_csv(FALCON_OUTPUT_DIR / "explanations.csv", index=Fal
(FALCON_OUTPUT_DIR / "explanations.json").write_text(
    json.dumps(results, ensure_ascii=False, indent=2), encoding="utf-8",
)
print(f"Caso {position}/20 generado.")

falcon_results = pd.DataFrame(results)
# Estos datos identifican el modelo y la configuración usados; sirven para i
(FALCON_OUTPUT_DIR / "generation_metadata.json").write_text(json.dumps({
    "model": FALCON_MODEL_NAME,
    "seed": FALCON_SEED,
    "generation_config": GENERATION_CONFIGS[SELECTED_CONFIG],

```

```
    "decoding": "greedy; temperatura efectiva 0",
}, ensure_ascii=False, indent=2), encoding="utf-8")
# Mostrar etiquetas, explicación y cita juntas para facilitar la revisión hu
with pd.option_context("display.max_colwidth", None, "display.max_rows", 25)
    display(falcon_results[[
        "test_row", "text", "category", "predicted_category", "reason",
        "evidence", "justification",
    ]])
```

Caso 1/20 generado.
Caso 2/20 generado.
Caso 3/20 generado.
Caso 4/20 generado.
Caso 5/20 generado.
Caso 6/20 generado.
Caso 7/20 generado.
Caso 8/20 generado.
Caso 9/20 generado.
Caso 10/20 generado.
Caso 11/20 generado.
Caso 12/20 generado.
Caso 13/20 generado.
Caso 14/20 generado.
Caso 15/20 generado.
Caso 16/20 generado.
Caso 17/20 generado.
Caso 18/20 generado.
Caso 19/20 generado.
Caso 20/20 generado.

test_row		text	category	predicted_
0	368	How do I unblock my card using the app?	card_not_working	pin
1	173	Where did this fee come from?	extra_charge_on_statement	card_payment_fee
2	2701	I transferred some money but it is yet to arrive.	balance_not_updated_after_bank_transfer	transfer_not_received_by
3	2604	how do VR cards work	get_disposable_virtual_card	getting_vir

test_row	text	category	predicted_c
4	2939 I will need to get a new card soon, how do I order?	card_about_to_expire	order_phys
5	2248 Is it possible for me to get money out in a different currency?	receiving_money	exchange
6	2523 My non-physical card will not work	virtual_card_not_working	card_not
7	2691 How long does a UK transfer take?	balance_not_updated_after_bank_transfer	transf
8	246 There is an incoming payment into my account, but it is deactivated. Will they still be processed?	fiat_currency_support	reverted_card_i
9	1381 How can I create many temporary cards daily?	disposable_card_limits	getting_sp

test_row		text	category	predicted_
10	673	Where is the money I topped off with?	pending_top_up	topping_up
11	2592	The see the rate applied to my transaction was wrong at the atm.	wrong_exchange_rate_for_cash_withdrawal	card_payment_wrong_exch
12	2711	My transfer is pending.	balance_not_updated_after_bank_transfer	pending
13	2606	I would like a temporary virtual card	get_disposable_virtual_card	getting_vir
14	1101	I think my statement has a fraudulent charge	card_payment_not_recognised	direct_debit_payment_not_re
15	1756	My card is being declined online. Could you tell me what might be broken or wrong with	declined_transfer	declined_card_

	test_row	text	category	predicted_
		the account?		
16	2341	What methods can I use to add money to my account?	transfer_into_account	topping_up
17	280	I need my card now!	card_delivery_estimate	ca
18	945	Can I get a copy of the card by email?	getting_virtual_card	order_phys
19	1517	I was charged on my account that shouldn't be there.	direct_debit_payment_not_recognised	card_payment_fee

4.5 Análisis y listado de las razones proporcionadas

La tabla muestra las veinte razones y justificaciones sugeridas por el LLM junto con la evidencia textual que citó. El recuento agrupa las razones para facilitar la lectura; describe únicamente esta muestra y las categorías indicadas en el prompt.

```
In [ ]: # Traducir los códigos previstos en el prompt; si aparece uno nuevo, conservarlo
REASON_LABELS_ES = {
    "semantic_overlap": "Solapamiento semántico",
    "negation_or_status": "Negación o estado de la operación",
    "ambiguity": "Ambigüedad de la consulta",
    "insufficient_evidence": "Información insuficiente",
    "unsupported_prediction": "Predicción sin apoyo textual",
}

falcon_results["razón"] = falcon_results["reason"].map(REASON_LABELS_ES).fillna("")
# Excluir razones vacías del recuento; no elimina esas filas del listado de falcon_results
falcon_reason_counts = (
    falcon_results.loc[falcon_results["reason"].astype(str).str.strip().ne("")].value_counts().rename_axis("razón").reset_index(name="casos")
)

print(f"Muestras con explicación solicitada: {len(falcon_results)}")
display(falcon_reason_counts)
```

```

# Mostrar los campos necesarios para comprobar si la explicación coincide co
with pd.option_context("display.max_colwidth", None, "display.max_rows", 25)
    display(falcon_results[[
        "test_row", "text", "category", "predicted_category", "razón",
        "evidence", "justification",
    ]])

# Guardar por separado el recuento agregado y el detalle de cada explicación
falcon_reason_counts.to_csv(FALCON_OUTPUT_DIR / "reason_frequencies.csv", in
falcon_results.to_csv(FALCON_OUTPUT_DIR / "explanations_reviewed.csv", index

```

Muestras con explicación solicitada: 20

	razón	casos
0	Información insuficiente	17
1	Solapamiento semántico	3

test_row		text	category	predicted_
0	368	How do I unblock my card using the app?	card_not_working	pin
1	173	Where did this fee come from?	extra_charge_on_statement	card_payment_fee
2	2701	I transferred some money but it is yet to arrive.	balance_not_updated_after_bank_transfer	transfer_not_received_by
3	2604	how do VR cards work	get_disposable_virtual_card	getting_vir

test_row		text	category	predicted_c
4	2939	I will need to get a new card soon, how do I order?	card_about_to_expire	order_phys
5	2248	Is it possible for me to get money out in a different currency?	receiving_money	exchange
6	2523	My non-physical card will not work	virtual_card_not_working	card_not
7	2691	How long does a UK transfer take?	balance_not_updated_after_bank_transfer	transf
8	246	There is an incoming payment into my account, but it is deactivated. Will they still be processed?	fiat_currency_support	reverted_card_i
9	1381	How can I create many temporary cards daily?	disposable_card_limits	getting_sp

test_row		text	category	predicted_
10	673	Where is the money I topped off with?	pending_top_up	topping_up
11	2592	The see the rate applied to my transaction was wrong at the atm.	wrong_exchange_rate_for_cash_withdrawal	card_payment_wrong_exch
12	2711	My transfer is pending.	balance_not_updated_after_bank_transfer	pending
13	2606	I would like a temporary virtual card	get_disposable_virtual_card	getting_vir
14	1101	I think my statement has a fraudulent charge	card_payment_not_recognised	direct_debit_payment_not_re
15	1756	My card is being declined online. Could you tell me what might be broken or wrong with	declined_transfer	declined_card_

	test_row	text	category	predicted_
		the account?		
16	2341	What methods can I use to add money to my account?	transfer_into_account	topping_up
17	280	I need my card now!	card_delivery_estimate	ca
18	945	Can I get a copy of the card by email?	getting_virtual_card	order_phys
19	1517	I was charged on my account that shouldn't be there.	direct_debit_payment_not_recognised	card_payment_fee

5. Conclusiones

El EDA mostró que BANKING77 contiene consultas cortas en inglés y 77 tipos de intención. También encontró diferencias en la cantidad de ejemplos de entrenamiento por tipo: algunas categorías tienen muchos menos ejemplos que otras, aunque la prueba está equilibrada con 40 consultas por categoría. Palabras como “not” y frases como “virtual card” aparecen con frecuencia; por eso se conservaron las consultas originales y no se quitaron palabras que pueden cambiar su significado.

DistilRoBERTa clasificó correctamente **92.18 %** de las **3,080** consultas de prueba y cometió **241 errores**. Es un buen resultado para este conjunto, pero no es perfecto: la confusión más frecuente fue entre `why_verify_identity` y `verify_my_identity`, con **9 casos**. Sus nombres e intención son cercanos, así que revisar ejemplos de este tipo puede ayudar a entender dónde se equivoca el modelo; el conteo por sí solo no confirma la causa.

Falcon analizó una muestra de **20 errores**. Propuso solapamiento de significado en **3 casos** y marcó **17** como información insuficiente. Esto muestra una limitación práctica: aunque se le pidan posibles razones, a menudo no ofrece una explicación concreta. Las que sí propone son

hipótesis basadas en el texto, no una descripción de cómo decidió DistilRoBERTa ni una causa comprobada del error.

En conjunto, los resultados sirven como punto de partida, no como garantía para consultas reales: el experimento usa textos en inglés de un conjunto específico, y la revisión del LLM cubre solo 20 errores. Para evaluar cambios con mayor confianza, conviene usar validación durante las pruebas y reservar un conjunto nuevo e independiente para la evaluación final.

```
In [ ]: # Convertir los conteos a texto para incluirlos en el resumen que se muestra
reason_listing = "; ".join(
    f"{row['razón']}: {int(row['casos'])}"
    for row in falcon_reason_counts.to_dict(orient="records")
) or "No se generó una razón estructurada"

# Reunir el tamaño de la muestra, el modelo y los parámetros junto con las r
falcon_summary = (
    "### Resumen de esta ejecución\n\n"
    f"Se solicitaron razones, citas y justificaciones posibles para **{len(f
    f"**{FALCON_MODEL_NAME}**. La generación fue greedy (temperatura efectiv
    f"**{GENERATION_CONFIGS[SELECTED_CONFIG]['max_new_tokens']** tokens y h
    f"**Razones sugeridas:** {reason_listing}.\n\n"
    "Son interpretaciones posibles basadas en el texto, no causas verificada
)
display(Markdown(falcon_summary))

# Guardar exactamente el texto mostrado para consultarlo o compartirlo fuera
(FALCON_OUTPUT_DIR / "conclusions.txt").write_text(falcon_summary, encoding=
```

Resumen de esta ejecución

Se solicitaron razones, citas y justificaciones posibles para **20** errores usando **tiuae/falcon-7b-instruct**. La generación fue greedy (temperatura efectiva 0), con máximo de **160** tokens y hasta cuatro oraciones de justificación.

Razones sugeridas: Información insuficiente: 17; Solapamiento semántico: 3.

Son interpretaciones posibles basadas en el texto, no causas verificadas del comportamiento del clasificador.

```
Out [ ]: 460
```

6. Mejoras técnicas propuestas

El modelo ya ofrece un buen resultado, así que conviene probar cambios pequeños y medir si realmente ayudan. El test actual se utilizó para revisar errores; para comparar nuevas versiones, usar validación y reservar un test nuevo e independiente para la comprobación final.

6.1 Propuestas en orden de prioridad

1. **Revisar los errores más repetidos.** Empezar por `why_verify_identity` → `verify_my_identity` (10 casos), `balance_not_updated_after_bank_transfer` → `transfer_timing` (5) y `get_disposable_virtual_card` → `getting_virtual_card` (5). Leer las consultas y comprobar si las etiquetas son correctas o si las categorías necesitan ejemplos más claros. Es la primera opción porque ayuda a entender el problema antes de cambiar el modelo.
2. **Comprobar si los pesos de clase ayudan.** Entrenar una versión sin pesos y compararla con la configuración actual, manteniendo los mismos datos y la misma validación. Comparar F1-macro (promedia el resultado de las clases por igual) y el recall de cada clase. Conservar los pesos solo si el resultado mejora sin perjudicar mucho a otras clases.
3. **Probar ejemplos de entrenamiento adicionales.** Crear unas pocas paráfrasis para las clases con pocos ejemplos o con confusiones frecuentes. Revisar cada frase a mano para asegurar que conserva el mismo significado, sobre todo la negación y el estado de una transferencia o tarjeta. Añadirlas solo al entrenamiento, nunca al test, y medir el resultado frente a la versión original.
4. **No aumentar todavía la longitud máxima por defecto.** Solo se truncó el 0.55% de las consultas de test. Revisar primero si esos casos explican errores; si no, mantener el límite actual. Si se prueba otro límite, elegirlo con validación y comparar también el tiempo de ejecución.

Estas pruebas no garantizan una mejora. Hacerlas de una en una permite identificar cuál aporta valor y evita complicar el sistema sin necesidad.